

Transcriptome pipelines

Peñaloza-Bojacá, G.F.; Villarreal, J.C.; Villarreal Lab.

2026-06-19

Contents

1	Introduction	6
1.1	Disclaimer	6
2	Transcriptome Analysis Workflow for Zamia RNA-seq Data	8
3	Files and Folders Organisation	9
4	Raw Reads	9
5	Trimming – Cleaning raw reads with Trimmomatic	10
5.1	Trimmomatic parameters used	10
5.2	SLURM script used for trimming	11
5.3	Trimming output files per sample	13
5.3.1	Example .log output file	13
5.3.2	Log summarization script	14
5.3.3	Trimming summary table	15
6	FastQC - Quality Control	16
6.1	FastQC parameters	16
6.2	SLURM script used for FastQC	17
6.3	FastQC output files per sample	18

7	Trinity: De novo transcriptome assembly	23
7.1	Trinity command parameters	23
7.2	SLURM script used for Trinity	24
7.3	Trinity output files per sample	25
7.4	Trinity Assembly Statistics	26
7.5	Trinity Co-assembly Using All Samples	27
7.5.1	Read concatenation	28
7.5.2	Trinity co-assembly command parameters	28
7.5.3	SLURM script used for Trinity co-assembly	28
7.5.4	Trinity co-assembly output files	30
8	Assembly quality assessment with TransRate	31
8.1	SLURM script used for TransRate	31
8.2	Transrate output files per sample	32
8.3	TransRate Assessment of the Global Trinity Assembly	35
8.3.1	TransRate command parameters	35
8.3.2	SLURM script used for global TransRate assessment	35
8.3.3	TransRate output files	37
9	BUSCO: Assembly Completeness Assessment	37
9.1	SLURM script used for BUSCO	38
9.2	BUSCO output files per sample	40
9.3	BUSCO summary	40
10	Sequence Similarity Searches: BLASTx and BLASTn	45
10.1	BLASTx: Similarity Search Against the NR Protein Database	45
10.1.1	BLASTx command parameters	45
10.1.2	SLURM script used for BLASTx analysis	46
10.1.3	BLASTx output files	48
10.2	BLASTn: Similarity Search Against the NT Database	48
10.2.1	BLASTn command parameters	48

10.2.2	SLURM script used for BLASTn analysis	49
10.2.3	BLASTn output files	50
11	Minimap2: Mapping assembled transcripts	50
11.1	Minimap2 command parameters	51
11.2	SLURM script used for Minimap2	51
11.3	Minimap2 output files per sample	53
12	BlobTools2: Taxonomic and Coverage-Based Assessment of Transcriptome Assemblies	54
12.1	BlobTools2 Database Creation	54
12.1.1	BlobTools2 create parameters	54
12.1.2	SLURM script used for BlobDir creation	55
12.1.3	BlobDir output files	56
12.2	Integration of BUSCO Completeness Metrics	57
12.2.1	BlobTools2 BUSCO integration parameters	57
12.2.2	SLURM script used for BUSCO integration	57
12.2.3	BUSCO variables integrated into BlobTools2	60
12.3	Integration of Read Coverage Information	60
12.3.1	BlobTools2 coverage integration parameters	61
12.3.2	SLURM script used for coverage integration	61
12.3.3	Coverage variables integrated into BlobTools2	62
12.4	Key BlobTools2 Variables for Transcriptome Assessment	62
12.4.1	Taxonomic Assignment	63
12.4.2	GC Content	63
12.4.3	Transcript Length	63
12.4.4	Read Coverage	63
12.4.5	BUSCO Status	63
12.5	BlobTools2 Output Files	63

13 BLASTx Against the Cycas Reference Proteome	64
13.1 BLASTx command parameters	64
13.2 Protein coverage assessment	65
13.3 SLURM script used for BLASTx against the Cycas reference proteome	65
13.4 Output files	66
14 Coverage-Based and Taxonomic Filtering of Transcript Assemblies	67
14.1 Filtering Transcripts by Reference Protein Coverage	67
14.1.1 Coverage filtering criterion	67
14.1.2 SLURM script used for coverage filtering	68
14.1.3 Output files	70
14.2 Taxonomic Assignment of High-Coverage Transcripts	70
14.2.1 BlobTools2 taxonomic assignment parameters	70
14.2.2 SLURM script used for taxonomic assignment	71
14.2.3 Output files	73
14.3 Extraction of Viridiplantae Transcripts	73
14.3.1 BlobTools2 filtering parameters	74
14.3.2 SLURM script used for Viridiplantae filtering	74
14.3.3 Output files	77
14.3.4 Final Output	77
15 TransRate Assessment of Filtered Transcriptomes	77
15.1 TransRate quality metrics	78
15.1.1 Interpretation of TransRate scores	79
15.2 SLURM script used for TransRate analysis	79
15.3 Output files	81
16 TransDecoder: Prediction of Coding Sequences	81
16.1 TransDecoder workflow	81
16.1.1 TransDecoder.LongOrfs	81
16.1.2 TransDecoder.Predict	82

16.2	Key output files	82
16.3	SLURM script used for TransDecoder	82
16.4	Summary statistics generated	84
16.5	Output files	85
17	TransDecoder Analysis of the Global Trinity Assembly	85
17.1	Purpose of the global analysis	85
17.2	TransDecoder workflow	85
17.2.1	TransDecoder.LongOrfs	86
17.2.2	TransDecoder.Predict	86
17.3	Summary statistics generated	86
17.4	SLURM script used for global TransDecoder analysis	86
17.5	Output files	88
17.5.1	Description of the main output files	88
18	Trinotate: Functional Annotation of the Global Transcriptome	88
18.1	Input files	89
18.2	Trinotate databases and annotation sources	89
18.3	Docker environment	89
18.4	Creation of the Trinotate database	90
18.4.1	EggNOG database issue	90
18.5	Initialization of the transcriptome database	90
18.6	Loading functional annotation results	90
18.6.1	BLASTX against SwissProt	90
18.6.2	BLASTP against SwissProt	91
18.6.3	Pfam domains	91
18.6.4	SignalP predictions	91
18.6.5	EggNOG annotations	91
18.7	Gene Ontology verification	92
18.8	Generation of the final annotation report	92
18.9	Output files	92

19 Salmon: Transcript Abundance Estimation	93
19.1 Input files	93
19.2 Output directory	93
19.3 Salmon workflow	94
19.4 Important parameters	94
19.5 SLURM script used for Salmon	94
19.6 Generation of abundance matrices	96
19.7 Output files	96
19.8 Interpretation of the main output files	97
20 Final Remarks and Future Analyses	97
20.1 Reference Transcriptome	97
20.2 Transcript Abundance Matrices	98
20.3 Future Analyses	98

1 Introduction

The goal of this tutorial is to document a step-by-step transcriptome analysis pipeline using a de novo assembly approach. This workflow has been developed for the *Zamia* RNA-seq dataset, and includes quality control, trimming, transcriptome assembly, annotation, and gene expression analysis.

All commands, scripts, and outputs have been organized to promote reproducibility and clarity. The pipeline is optimized for SLURM-based HPC environments, but can be adapted for other systems.

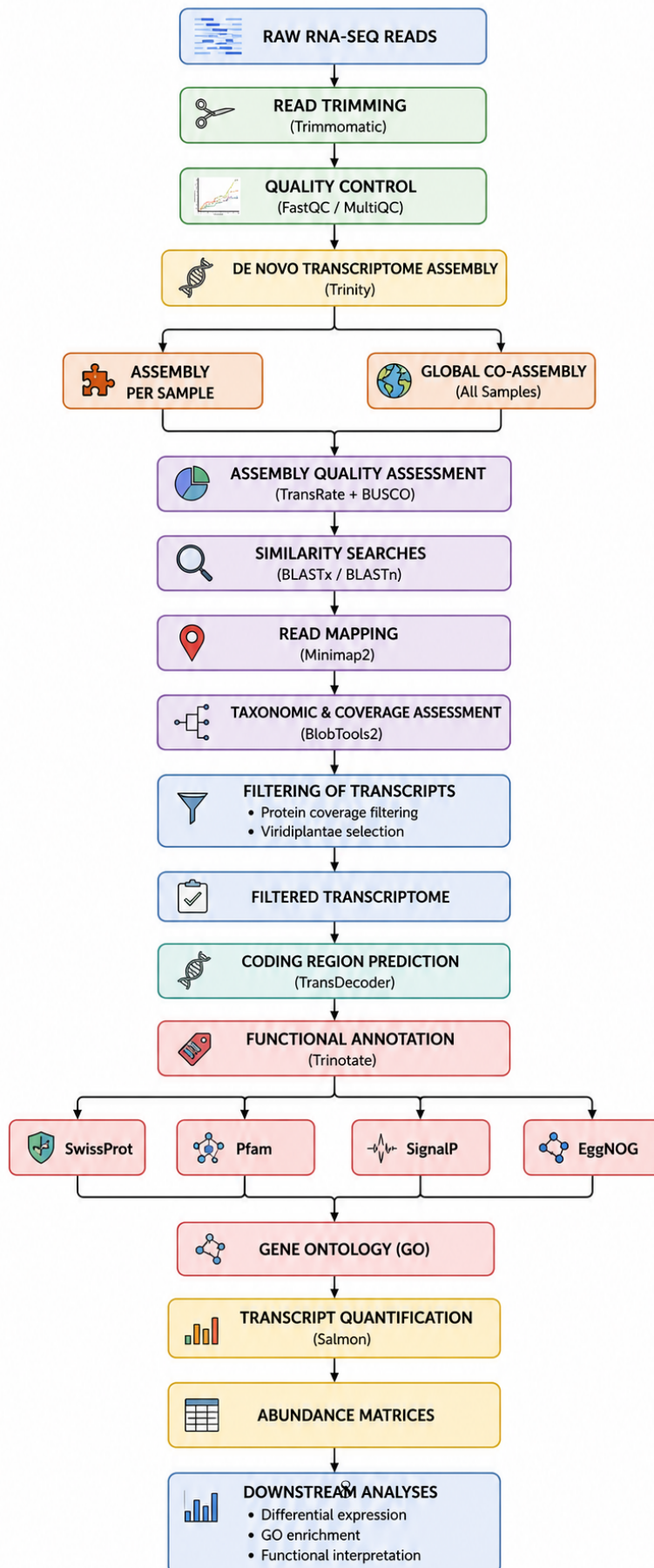
1.1 Disclaimer

This document provides an example workflow for transcriptome processing and analysis based on the tools, datasets, and objectives used in this project. It is intended as a practical guide rather than a definitive protocol.

Bioinformatics pipelines are continuously evolving, and alternative approaches, parameters, and software may be equally valid depending on the research question and available resources. Users are encouraged to consult the official documentation, recent literature, and software repositories for updates, best practices, and new features.

All scripts presented here are open and can be modified, adapted, and improved according to the needs of each project. Parts of the code, documentation, and workflow descriptions were developed with the assistance of artificial intelligence tools and subsequently reviewed and adapted for this analysis.

2 Transcriptome Analysis Workflow for Zamia RNA-seq Data



3 Files and Folders Organisation

Note: it is not advisable to change folder names once you've started processing.

We used WinSCP as SFTP. WinSCP is an open source free SFTP client for Windows. An SFTP client is a software which uses the SFTP protocol to transfer files securely to and from a remote computer.

Take your time to choose meaningful and structured names. A good folder name should help you quickly identify:

- the type of analysis included (e.g., raw reads, QC, assembly)
- the organism or project (e.g., *Zamia*)
- the type of molecular data (e.g., RNA, DNA)

Recommended directory structure

- 00_data/ – Reference files, genome indices, annotations
- 01_raw_reads_Zamia_RNA/ – Raw RNA-seq reads (FASTQ files)
- 02_clean_reads_Zamia_RNA_trimmomatic/ – Cleaned reads after trimming
- 03_fastqc_Zamia_RNA_clean/ – FastQC and MultiQC results for cleaned reads
- 04_trinity_Zamia_RNA/ – De novo transcriptome assembly (Trinity output)
- 05_busco_results_Zamia_RNA/ – Assembly completeness assessment with BUSCO

Alternative folder structure example

4 Raw Reads

The first step in the pipeline is to inspect and verify the structure of the raw sequence data. For this project, paired-end FASTQ files were generated from RNA extracted from *Zamia*. Files are stored in the 00_raw_reads_Zamia_RNA/ folder.

The raw reads are organized in one folder per sample and include two paired-end FASTQ files:

```
00_raw_reads_Zamia_RNA/  
  Raw-HM-JCV23-0101/  
    Raw-HM-JCV23-0101_R1_paired.fq.gz  # forward reads (R1)  
    Raw-HM-JCV23-0101_R2_paired.fq.gz  # reverse reads (R2)
```

These files are gzip-compressed (.fq.gz) and follow a consistent naming pattern for compatibility with batch processing scripts.

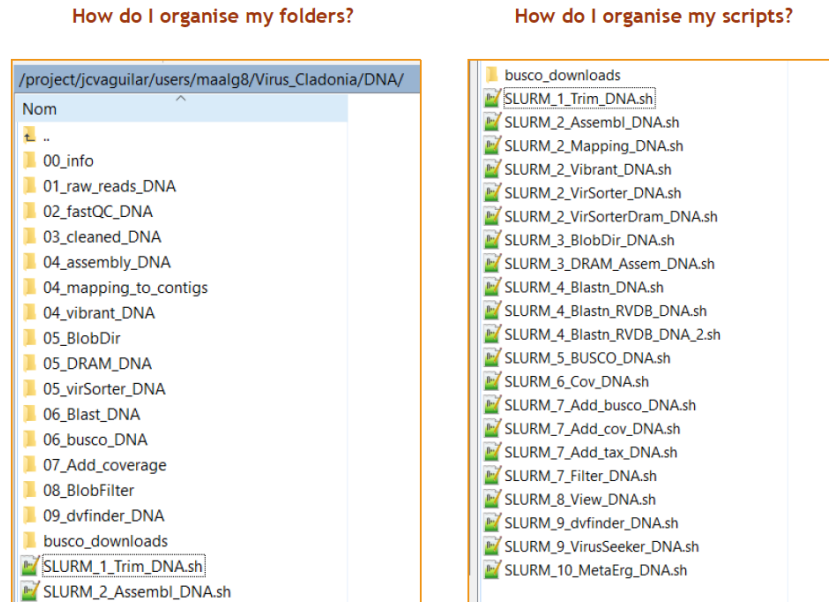


Figure 1: Example of a folder structure used in the genome pipeline, adapted from Alonso Garcia (2021).

5 Trimming – Cleaning raw reads with Trimmomatic

Trimming is an essential step to remove adapter sequences, low-quality bases, and very short reads from raw data. For this project, trimming was performed using **Trimmomatic** on paired-end reads.

For more information, see Trimmomatic manual.

Each input sample produced:

- Two **clean paired-end reads** (`*_R1_paired.fq.gz`, `*_R2_paired.fq.gz`)
- Two **unpaired reads** (`*_R1_unpaired.fq.gz`, `*_R2_unpaired.fq.gz`)
- A log file (`.log`) containing quality and trimming statistics

The output files were stored in:

`01_clean_reads_Zamia_RNA_trimmomatic/`

5.1 Trimmomatic parameters used

The following parameters were used in the trimming step to ensure high-quality reads for transcriptome assembly:

```
ILLUMINACLIP:<adapters>:2:30:10
LEADING:3
TRAILING:3
SLIDINGWINDOW:4:15
MINLEN:100
```

Parameter explanations:

ILLUMINACLIP::2:30:10:

Removes adapter sequences from reads.

- 2: Maximum mismatches allowed in adapter alignment
- 30: Minimum alignment score to remove a paired-end read
- 10: Minimum length of adapter match for single-end trimming

LEADING:3:

Removes low-quality bases from the start of each read (Phred score < 3).

TRAILING:3:

Removes low-quality bases from the end of each read (Phred score < 3).

SLIDINGWINDOW:4:15:

Performs quality trimming within a 4-base sliding window. Trimming starts when the average quality in the window drops below 15.

MINLEN:100:

Discards any read that becomes shorter than 100 nucleotides after trimming. This ensures that only reads of sufficient length are retained for downstream mapping and assembly.

These settings provide a good balance between removing sequencing noise and retaining informative RNA-seq reads, especially for **Illumina paired-end data of 150 bp**.

5.2 SLURM script used for trimming

The following script was used to trim all samples using a SLURM job array.

SLURM script: 01_A_Trim_Zamia_RNA.sh

```
#!/bin/bash
#SBATCH --partition=BRYOFIT
#SBATCH -J TrimArray_Zamia
#SBATCH -o Zamia_Trim_RNA_%A_%a.out
#SBATCH -e Zamia_Trim_RNA_%A_%a.err
#SBATCH -p medium
#SBATCH --array=1-12
#SBATCH --mail-type=ALL
#SBATCH --mail-user=gfpeb@ulaval.ca
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=10
#SBATCH --mem-per-cpu=4G
#SBATCH --time=24:00:00

module load trimmomatic/0.36
```

```

# --- Configuración ---
base_dir="/project/jcvaguilar/users/jcvil9/Gab_Zamia_Transcri"
trimmomatic_jar="${base_dir}/Programas/trimmomatic_0.36/trimmomatic-0.36.jar"
adapters="${base_dir}/Programas/trimmomatic_0.36/adapters/TruSeq3-PE.fa"
input_parent="${base_dir}/00_raw_reads_Zamia_RNA"
output_parent="${base_dir}/01_clean_reads_Zamia_RNA_trimmomatic"

# Obtener el nombre de muestra desde la lista
sample_name=$(sed -n "${SLURM_ARRAY_TASK_ID}p" ${base_dir}/samples_list.txt)
sample_dir="${input_parent}/Raw-HM-${sample_name}"

# Verifica si la muestra existe
if [[ ! -d "$sample_dir" ]]; then
    echo "Carpeta no encontrada: $sample_dir"
    exit 1
fi

# Archivos de entrada
R1="${sample_dir}/${sample_name}_R1.fastq.gz"
R2="${sample_dir}/${sample_name}_R2.fastq.gz"

# Crear carpeta de salida
output_dir="${output_parent}/${sample_name}"
mkdir -p "$output_dir"

# Archivos de salida
R1_paired="${output_dir}/${sample_name}_R1_paired.fastq.gz"
R1_unpaired="${output_dir}/${sample_name}_R1_unpaired.fastq.gz"

R2_paired="${output_dir}/${sample_name}_R2_paired.fastq.gz"
R2_unpaired="${output_dir}/${sample_name}_R2_unpaired.fastq.gz"

log_file="${output_dir}/${sample_name}_trimmomatic.log"

# Ejecutar Trimmomatic
echo "Procesando $sample_name..."
java -jar "$trimmomatic_jar" PE -phred33 \
    -threads 10 \
    "$R1" "$R2" \
    "$R1_paired" "$R1_unpaired" \
    "$R2_paired" "$R2_unpaired" \
    ILLUMINACLIP:"$adapters":2:30:10 \
    LEADING:3 TRAILING:3 SLIDINGWINDOW:4:15 MINLEN:100 \
    &> "$log_file"

echo "Finalizado $sample_name"

```

5.3 Trimming output files per sample

The output files will be stored in a new folder named *01_clean_reads_Zamia_RNA_trimmomatic*, organized into subdirectories by sample, each containing the trimmed read files and associated logs.

```
01_clean_reads_Zamia_RNA_trimmomatic/
  JCV23-0101/
    JCV23-0101_R1_paired.fq.gz      # Forward reads after trimming
    JCV23-0101_R1_unpaired.fq.gz   # Unpaired forward reads
    JCV23-0101_R2_paired.fq.gz     # Reverse reads after trimming
    JCV23-0101_R2_unpaired.fq.gz   # Unpaired reverse reads
    JCV23-0101_trimmomatic.log      # Log file with trimming summary
  JCV23-0102/
    ...
```

Each sample is stored in its own subfolder to simplify downstream processing and maintain a clean directory structure.

5.3.1 Example .log output file

The Trimmomatic log files report the number of paired-end reads processed and how many of them were retained after trimming. Below is an example summary for one sample:

```
TrimmomaticPE: Started with arguments:
Using PrefixPair: 'TACACTCTTTCCCTACACGACGCTCTTCCGATCT' and
  ↳ 'GTGACTGGAGTTCAGACGTGTGCTCTTCCGATCT'
ILLUMINACLIP: Using 1 prefix pairs, 0 forward/reverse sequences, 0 forward only
  ↳ sequences, 0 reverse only sequences
Input Read Pairs: 31773254 Both Surviving: 29326932 (92.30%) Forward Only
  ↳ Surviving: 2230442 (7.02%) Reverse Only Surviving: 88908 (0.28%) Dropped:
  ↳ 126972 (0.40%)
TrimmomaticPE: Completed successfully
```

- **Input Read Pairs:** The total number of read pairs provided to Trimmomatic.
- **Both Surviving:** Pairs where both reads passed quality and length filters — retained as paired reads.
- **Forward/Reverse Only Surviving:** Cases where only one read from the pair survived. These reads are kept in the unpaired output files.
- **Dropped:** Read pairs where neither read passed the filters — these are discarded.

In this example, **92.3%** of read pairs were successfully retained as clean paired-end reads, indicating good initial data quality. Only **0.4%** of pairs were discarded entirely, which is a very low loss rate and suggests minimal adapter contamination or poor-quality bases.

5.3.2 Log summarization script

To summarize the number of reads retained and discarded during the trimming process, the following script *01_B_Trim_Summary_Zamia_RNA.sh* was used. It parses all Trimmomatic *.log* files and extracts key statistics such as the number of surviving read pairs, unpaired reads, and dropped pairs.

SLURM script: *01_B_Trim_Summary_Zamia_RNA.sh*

```
#!/bin/bash

#SBATCH -J TrimSum_Zamia
#SBATCH -p small
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=2G
#SBATCH --mail-type=ALL
#SBATCH --mail-user=gfpeb@ulaval.ca
#SBATCH --time=1:00:00

#####
# Configuración general

base_dir="/project/jcvaguilar/users/jcvil9/Gab_Zamia_Transcri"
samples_list="${base_dir}/samples_list.txt"
output_dir="${base_dir}/01_clean_reads_Zamia_RNA_trimmomatic"
resumen_file="${output_dir}/Trimmomatic_summary.csv"

#####
# Encabezado

echo "Sample,Input_Pairs,Both_Surviving,Forward_Only,Reverse_Only,Dropped" >
  ↪ "$resumen_file"

#####
# Procesar cada muestra

while IFS= read -r sample; do
    log="${output_dir}/${sample}/${sample}_trimmomatic.log"

    if [[ -f "$log" ]]; then
        # Extraer la línea que contiene los resultados
        stats_line=$(grep "Input Read Pairs" "$log")

        if [[ -n "$stats_line" ]]; then
            # Extraer valores usando expresiones regulares
            input=$(echo "$stats_line" | sed -n 's/.*Input Read Pairs:
            ↪ \([0-9]*\)*/\1/p')
            both=$(echo "$stats_line" | sed -n 's/.*Both Surviving:
            ↪ \([0-9]*\)*/\1/p')
```

```

forward=$(echo "$stats_line" | sed -n 's/.*Forward Only Surviving:
↪ \([0-9]*\)*/\1/p')
reverse=$(echo "$stats_line" | sed -n 's/.*Reverse Only Surviving:
↪ \([0-9]*\)*/\1/p')
dropped=$(echo "$stats_line" | sed -n 's/.*Dropped:
↪ \([0-9]*\)*/\1/p')
echo "$sample,$input,$both,$forward,$reverse,$dropped" >>
↪ "$resumen_file"

else
echo "$sample,ERROR: No data found" >> "$resumen_file"
fi
else
echo "$sample,ERROR: Log file not found" >> "$resumen_file"
fi
done < "$samples_list"
echo "Resumen generado en: $resumen_file"

```

5.3.3 Trimming summary table

The following table summarizes the results of the trimming step for all samples. It includes the total number of input read pairs, the percentage of pairs that survived, and those that were partially or fully discarded.

```

library(readr)
library(dplyr)
library(knitr)

# Leer el archivo CSV desde tu computadora
trim_summary <- readr::read_csv(file.path(FIG, "Trimmomatic_summary.csv"))

# Mostrar tabla ordenada por muestra
trim_summary <- trim_summary %>% arrange(Sample)

# Mostrar tabla
kable(trim_summary, caption = "Table 2. Trimming summary for each sample")

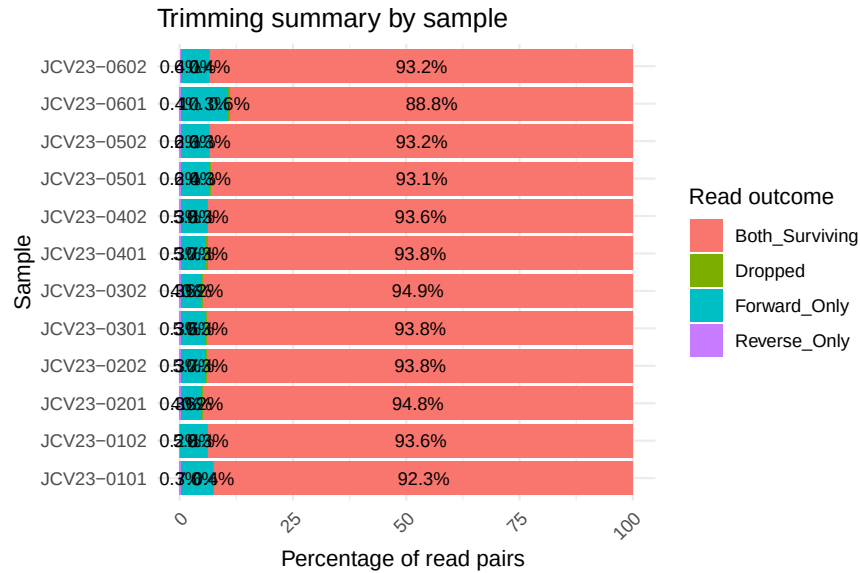
```

Table 1: Table 2. Trimming summary for each sample

Sample	Input_Pairs	Both_Surviving	Forward_Only	Reverse_Only	Dropped
JCV23-0101	31773254	29326932	2230442	88908	126972
JCV23-0102	36485938	34163732	2133170	83887	105149
JCV23-0201	32697196	31007931	1508918	104751	75596
JCV23-0202	33684743	31596155	1914412	87691	86485
JCV23-0301	32871179	30831693	1847803	92105	99578
JCV23-0302	36643793	34760975	1697357	98848	86613

Sample	Input_Pairs	Both_Surviving	Forward_Only	Reverse_Only	Dropped
JCV23-0401	30131960	28252449	1705129	92830	81552
JCV23-0402	32456879	30377489	1897022	84033	98335
JCV23-0501	35067922	32650201	2236630	80287	100804
JCV23-0502	36945684	34429194	2318389	91581	106520
JCV23-0601	28889052	25648588	2970012	101352	169100
JCV23-0602	33737733	31452369	2027863	137290	120211

Trimming summary plot



6 FastQC - Quality Control

After trimming, we assessed the quality of the cleaned reads using **FastQC**, a tool designed to provide quick diagnostics of sequencing data. FastQC generates per-sample HTML reports summarizing base quality, adapter content, GC distribution, and more.

For more information, see FastQC

The analysis was performed on the cleaned paired-end reads (`*_R1_paired.fq.gz` and `*_R2_paired.fq.gz`) located in the folder `01_clean_reads_Zamia_RNA_trimmomatic/`.

The output files were saved in a dedicated folder:

`02_fastqc_Zamia_clean`

6.1 FastQC parameters

FastQC was run with default parameters. The only specified option was the output directory:

By default, FastQC performs the following checks:

- Per base sequence quality
- Per sequence quality scores
- Per base sequence content
- Per base GC content
- Per sequence GC content
- Per base N content
- Sequence length distribution
- Sequence duplication levels
- Overrepresented sequences
- Adapter content

No custom adapters or contaminants were specified, and no filtering or trimming was applied at this step — FastQC is purely diagnostic and does not modify the input data.

6.2 SLURM script used for FastQC

The following script was used to FastQC all samples using a SLURM job array.

SLURM script: 02_A_fastqc_Zamia_RNA.sh

```
#!/bin/bash

#SBATCH --partition=BRYOFIT
#SBATCH -J fastqc_zamia_array
#SBATCH -o fastqc_zamia_%A_%a.out
#SBATCH -e fastqc_zamia_%A_%a.err
#SBATCH -p medium
#SBATCH --array=1-12
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=2
#SBATCH --mem-per-cpu=2G
#SBATCH --time=01:00:00
#SBATCH --mail-type=ALL
#SBATCH --mail-user=gfpeb@ulaval.ca

module load fastqc/0.11.8

# Configuración de directorios
base_dir="/project/jcvaguilar/users/jcvil9/Gab_Zamia_Transcri"
input_dir="${base_dir}/01_clean_reads_Zamia_RNA_trimmomatic"
output_base="${base_dir}/02_fastqc_Zamia_clean"
samples_list="${base_dir}/samples_list.txt"

# Obtener nombre de muestra
sample_name=$(sed -n "${SLURM_ARRAY_TASK_ID}p" "$samples_list")
sample_folder="${input_dir}/${sample_name}"

# Verificar si existe la carpeta de la muestra
```

```

if [[ ! -d "$sample_folder" ]]; then
    echo "Carpeta no encontrada: $sample_folder"
    exit 1
fi

# Crear carpeta de salida para la muestra
sample_out="${output_base}/${sample_name}"
mkdir -p "$sample_out"

# Ejecutar FastQC para cada archivo .fastq.gz de la muestra

for file in "$sample_folder"/*.fastq.gz; do
    echo "Procesando $file ? $sample_out"
    fastqc -o "$sample_out" "$file"
done

echo "FastQC finalizado para muestra $sample_name"

```

6.3 FastQC output files per sample

The output files will be stored in a new folder named *02_fastqc_Zamia_clean*, organized into subdirectories by sample.

```

02_fastqc_Zamia_clean/
  JCV23-0101
    JCV23-0101_R1_paired_fastqc.html
    JCV23-0101_R1_paired_fastqc.zip
    JCV23-0101_R2_paired_fastqc.html
    JCV23-0101_R2_paired_fastqc.zip
    ...
  JCV23-0102
    ...

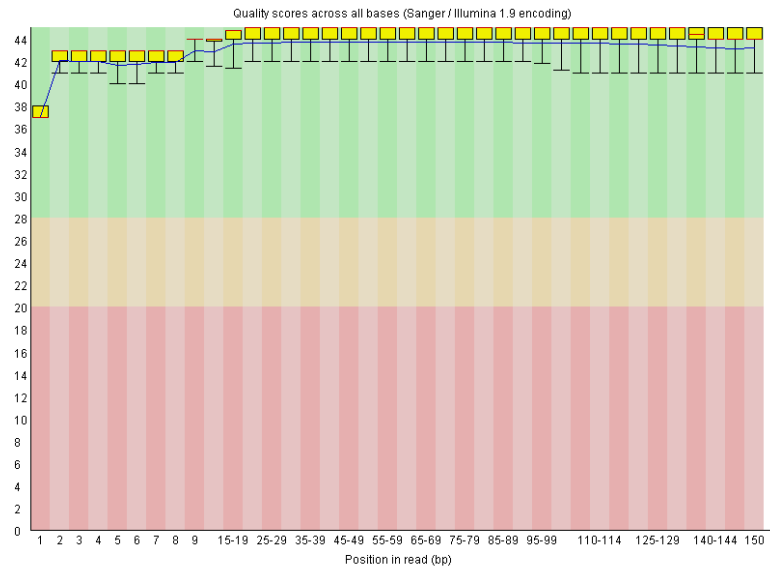
```

- An **.html** report that provides a detailed visual summary of read quality and sequence statistics.
- A **.zip** archive containing raw data files used to generate the report (e.g., tables, images, summary text).

The **.html** files can be opened directly in a web browser to inspect per-base quality scores, GC content, adapter contamination, sequence length distribution, and other metrics. These diagnostics help assess whether the reads are suitable for downstream processing (e.g., transcriptome assembly or quantification).

Important Metrics

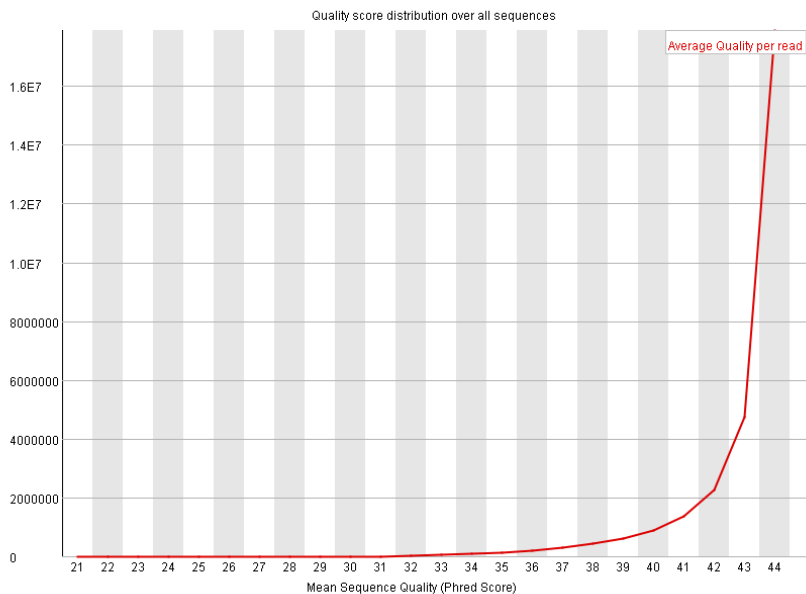
✓ Per base sequence quality



Per base sequence quality

- Assesses the average quality score at each base position in the read.
- Look for Phred scores >30 across most positions. A slight drop at the end is normal, but sharp declines may require trimming.
- This is the **most critical plot** to determine base-by-base quality.

✓ Per sequence quality scores

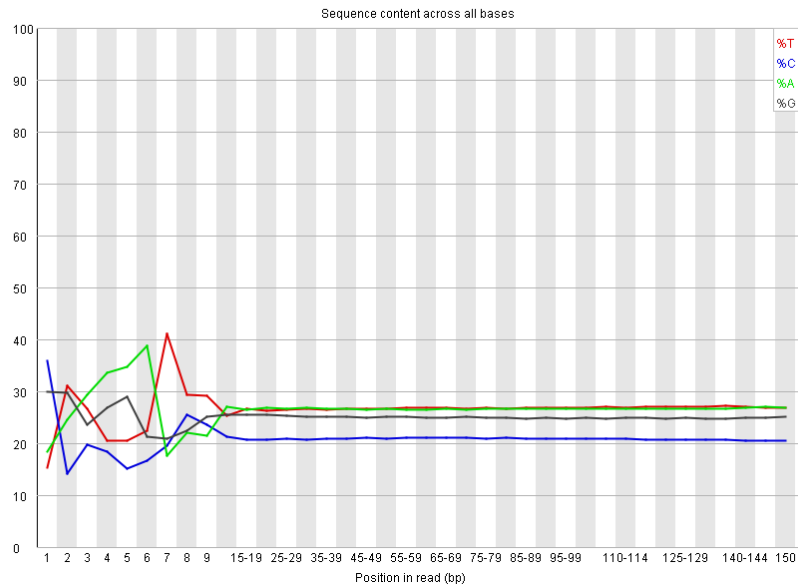


Per sequence quality scores

- Displays the distribution of average quality scores per read.

- Single sharp peak in the high-quality range is ideal. A bimodal or flat distribution may indicate library issues.

❌ Per base sequence content

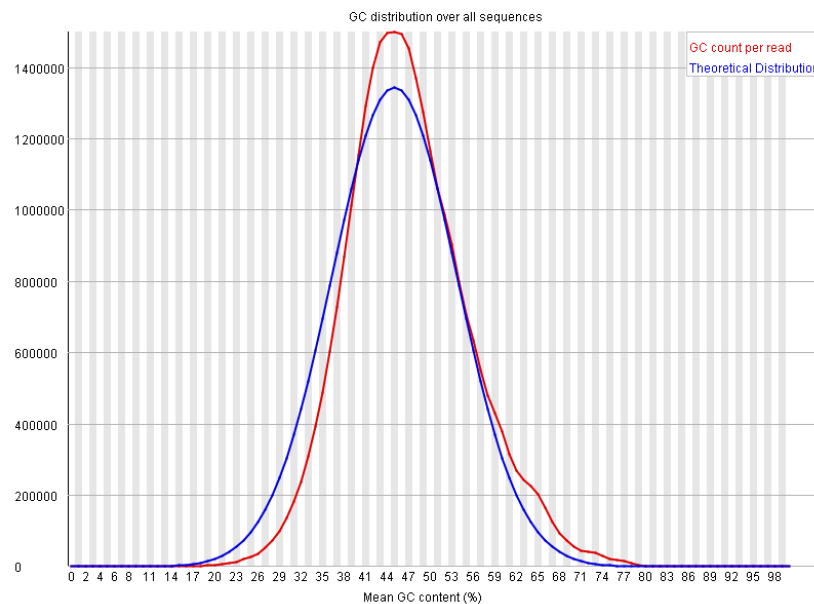


Per base sequence content

- Shows the proportion of A, T, G, and C at each base position.
- For RNA-seq, deviations from uniformity are expected due to transcript biases.
- Often marked as [FAIL], but this is **not necessarily a problem** in RNA-seq.

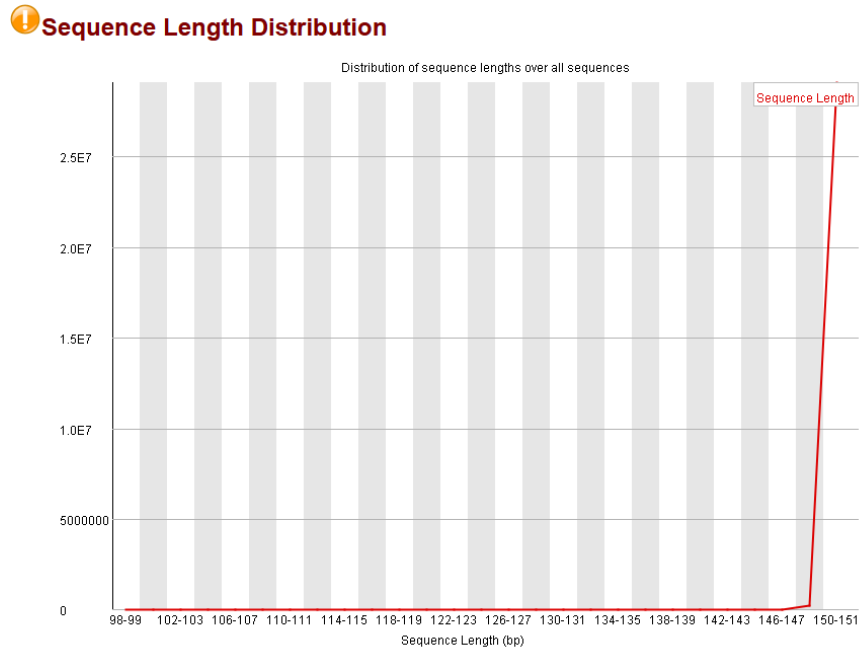
Useful Additional Metrics

⚠️ Per sequence GC content



GC content

- Displays the distribution of GC content across reads.
- Single bell-shaped curve centered around the expected GC content is ideal.
- Bimodal or skewed distributions may suggest contamination.

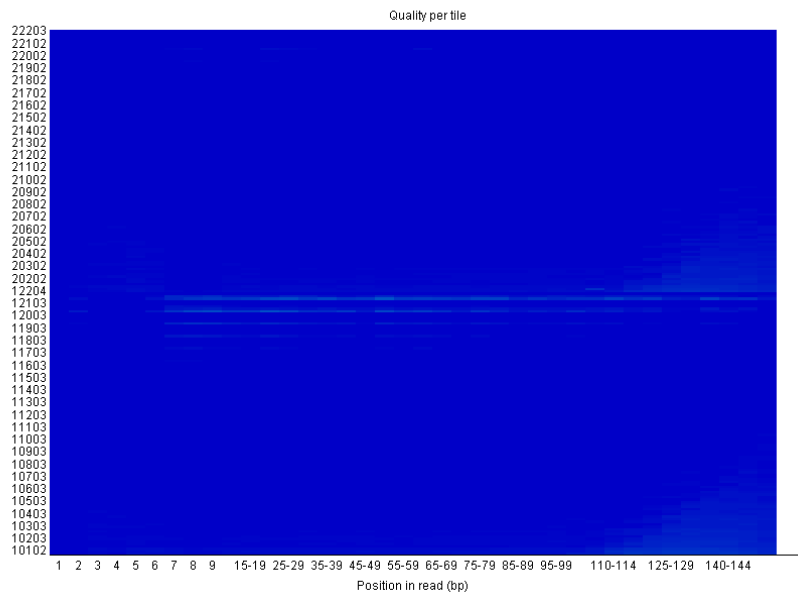


Sequence length distribution

- Indicates the length of reads in your dataset.
- Most reads should have the same length (e.g., 150 bp), unless trimming or filtering was applied.

Secondary Metrics (review if flagged)

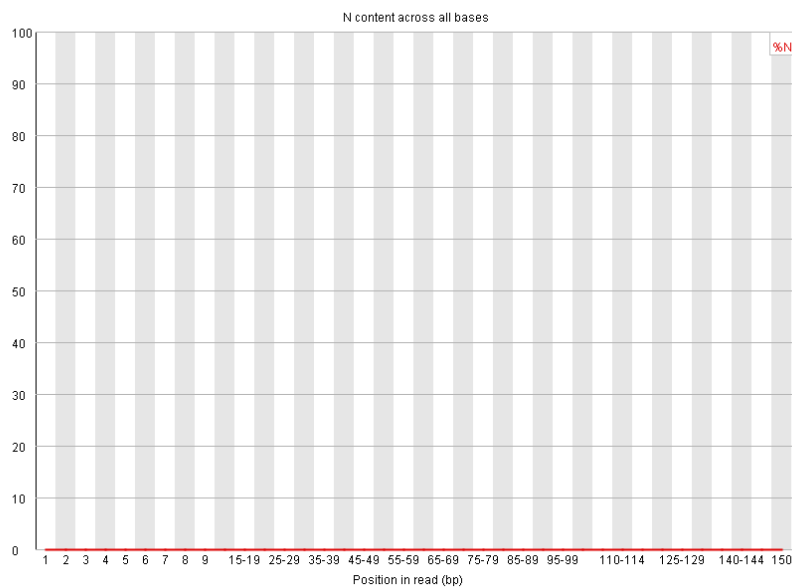
✓ Per tile sequence quality



Per tile sequence quality

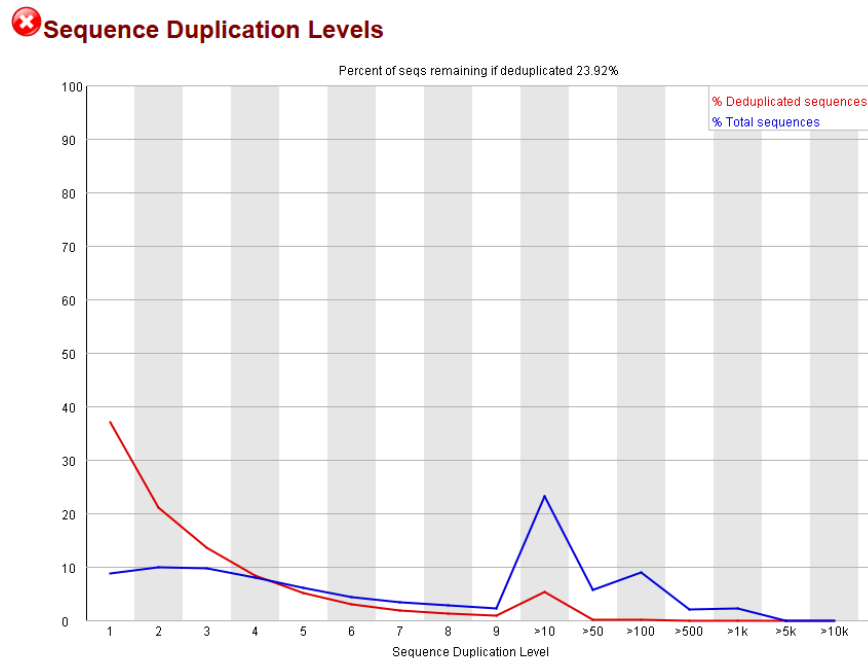
- Detects systematic problems across different areas (“tiles”) of the flowcell.
- Usually not an issue, unless you observe significant [FAIL]s.

✓ Per base N content



Per base N content

- Measures the proportion of unidentified bases (N) at each position.
- Should be near 0%. Peaks may indicate sequencing issues.



Sequence duplication levels

- Evaluates how many reads are duplicated.
- Some duplication is normal in RNA-seq due to highly expressed genes.
- High duplication (>50%) may suggest PCR artifacts or low library complexity.

7 Trinity: De novo transcriptome assembly

Trinity is a widely used tool for *de novo* transcriptome assembly from RNA-seq data. It reconstructs full-length transcripts by combining short reads into contigs using a modular approach: **Inchworm**, **Chrysalis**, and **Butterfly**. This step is critical for building a reference transcriptome in non-model organisms, such as *Zamia*.

Trinity works best on **high-quality, paired-end reads** that have already been trimmed (e.g. with Trimmomatic) and quality-checked (e.g. with FastQC).

For more information, see Trinity

The analysis was performed on the cleaned paired-end reads (`*_R1_paired.fq.gz` and `*_R2_paired.fq.gz`) located in the folder `01_clean_reads_Zamia_RNA_trimmomatic/`.

The output files were saved in a dedicated folder:

`03_trinity_separated`

7.1 Trinity command parameters

The Trinity assembly was executed using the following key parameters:

- `--seqType fq`: Specifies that input files are in FASTQ format.

- `--left` and `--right`: Paired-end reads used for assembly.
- `--max_memory 80G`: Limits RAM usage to 80 GB, which aligns with the SLURM job allocation.
- `--CPU 4`: Trinity ran on 4 threads, balancing performance and cluster availability.
- `--output`: Output directory specific to each sample.
- `--full_cleanup`: Removes intermediate files after completion to reduce disk space.
- `> "$log_file" 2>&1`: Redirects both standard output and error messages to a log file for later inspection.

These parameters ensure that the assembly is efficient, reproducible, and does not overload the compute node.

7.2 SLURM script used for Trinity

We used a SLURM array job to run Trinity in parallel for each sample.

SLURM script: 03_C_Trinity_Zamia_RNA.sh

```
#!/bin/bash

#SBATCH -J Tri_Zam_RNA
#SBATCH -o Trinity_Zamia_%j.out
#SBATCH -e Trinity_Zamia_%A_%a.err
#SBATCH -p medium
#SBATCH --array=6
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --mem=80G
#SBATCH --time=6-00:00:00
#SBATCH --mail-type=ALL
#SBATCH --mail-user=gfpeb@ulaval.ca

source /prg/miniconda/3/etc/profile.d/conda.sh
conda activate /home/jcvil9/.conda/envs/trinity_env

# Rutas
base_dir="/project/jcvaguilar/users/jcvil9/Gab_Zamia_Transcri"
samples_list="${base_dir}/samples_list.txt"
input_base="${base_dir}/01_clean_reads_Zamia_RNA_trimmomatic"
output_base="${base_dir}/03_trinity_separated"

# Obtener muestra actual
sample=$(sed -n "${SLURM_ARRAY_TASK_ID}p" "$samples_list" | tr -d '[:space:]')
fecha=$(date +%Y%m%d)
output_dir="${output_base}/${sample}_${fecha}_trinity"
log_file="${output_dir}/${sample}_${fecha}_trinity.log"
```



```

# Archivos de entrada
left_reads="${input_base}/${sample}/${sample}_R1_paired.fastq.gz"
right_reads="${input_base}/${sample}/${sample}_R2_paired.fastq.gz"

# Verificar existencia
if [[ ! -f "$left_reads" || ! -f "$right_reads" ]]; then
    echo " FASTQ no encontrados para muestra $sample" >&2
    exit 1
fi

mkdir -p "$output_dir"

# Ejecutar Trinity directamente
echo "Ejecutando Trinity para $sample..."
Trinity \
    --seqType fq \
    --left "$left_reads" \
    --right "$right_reads" \
    --max_memory 80G \
    --CPU 4 \
    --output "$output_dir" \
    --full_cleanup \
    > "$log_file" 2>&1

echo "Trinity finalizado para $sample"
conda deactivate

```

7.3 Trinity output files per sample

The output files will be stored in a new folder named 03_trinity_separated, organized into subdirectories by sample.

```

03_trinity_separated/
  JCV23-0101_20250617
  JCV23-0102_20250617
  JCV23-0201_20250617
  ...

```

/project/jcvaguilar/users/jcvil9/Gab_Zamia_Transcri/03_trinity_separated/JCV23-0101_20250617_trinity/	
Nome	Tamanho
...	
__salmon_filt.chkpts	
chrysalis	
insilico_read_normalization	
read_partitions	
salmon_outdir	
Trinity.tmp.fasta.salmon.idx	
both.fa	6.273.306 ...
both.fa.ok	0 KB
both.fa.read_count	1 KB
inchworm.DS.fa	352.937 KB
inchworm.DS.fa.finished	0 KB
inchworm.kmer_count	1 KB
JCV23-0101_20250617_trinity.log	9.212 KB
JCV23-0101_20250617_trinity.Trinity.fasta	168.141 KB
JCV23-0101_20250617_trinity.Trinity.fasta.gene_trans_map	12.057 KB
jellyfish.kmers.25.asm.fa	6.955.654 ...
jellyfish.kmers.25.asm.fa.histo	13 KB
left.fa.ok	0 KB
partitioned_reads.files.list	31.701 KB
partitioned_reads.files.list.ok	0 KB
pipeliner.134911.cmds	1 KB
recursive_trinity.cmds	108.736 KB
recursive_trinity.cmds.completed	108.736 KB
recursive_trinity.cmds.ok	0 KB
right.fa.ok	0 KB
scaffolding_entries.sam	17.030.96...
Trinity.tmp.fasta	245.255 KB

7.4 Trinity Assembly Statistics

The following table shows the basic statistics of the transcriptome assembly produced by Trinity, obtained with the script `TrinityStats.pl`.

The following summary shows the basic assembly statistics generated by `TrinityStats.pl` for the sample JCV23-0101, including the number of genes and transcripts, GC content, N statistics, contig lengths, and total assembled bases.

```
#####
## Counts of transcripts, etc.
#####
Total trinity 'genes': 209841
Total trinity transcripts: 260669
Percent GC: 40.80

#####
Stats based on ALL transcript contigs:
#####

Contig N10: 4185
Contig N20: 3054
Contig N30: 2338
Contig N40: 1799
Contig N50: 1265

Median contig length: 271
```

```
Average contig: 586.99
Total assembled bases: 153009149
```

```
#####
## Stats based on ONLY LONGEST ISOFORM per 'GENE':
#####
```

```
Contig N10: 3561
Contig N20: 2379
Contig N30: 1602
Contig N40: 784
Contig N50: 418
```

```
Median contig length: 260
Average contig: 435.42
Total assembled bases: 91368473
```

These metrics provide a general overview of transcriptome assembly quality:

Number of transcripts and genes: A high number (260k transcripts from 209k genes) is expected for *de novo* assemblies in non-model species, though redundancy can occur.

GC content (40.80%): Within the typical range for plant transcriptomes (usually between 40–45%), suggesting no major contamination.

N50 and contig length: N50 of 1,265 bp (all contigs) and 418 bp (longest isoforms only) are moderate values. A higher N50 generally reflects longer, more complete transcripts.

Total assembled bases: ~153 Mbp (all contigs) → a good indication of coverage. ~91 Mbp (longest isoforms) is more useful for downstream gene expression analysis.

Tip: A high average contig length and N50, along with reasonable GC content and number of transcripts, support a reliable assembly. However, functional completeness (BUSCO) and read support (TransRate) are also important.

7.5 Trinity Co-assembly Using All Samples

In addition to the individual transcriptome assemblies generated for each sample, a global *de novo* transcriptome assembly was performed using all cleaned RNA-seq libraries combined into a single dataset. This co-assembly strategy increases sequencing depth, improves transcript recovery, and provides a comprehensive reference transcriptome for downstream analyses such as BUSCO, BLASTx, Minimap2, BlobTools2, contamination filtering, and functional annotation.

The analysis was performed using all cleaned paired-end reads (*_R1_paired.fastq.gz and *_R2_paired.fastq.gz) located in the folder 01_clean_reads_Zamia_RNA_trimmomatic/.

The output files were saved in a dedicated folder:

```
03_trinity_separated/Zamia_all_YYYYMMDD_trinity
```

7.5.1 Read concatenation

Prior to assembly, all cleaned paired-end reads were concatenated into two compressed FASTQ files:

- `all_samples_R1.fastq.gz`
- `all_samples_R2.fastq.gz`

This procedure allowed Trinity to reconstruct a reference transcriptome containing transcripts expressed across all samples included in the study.

7.5.2 Trinity co-assembly command parameters

The Trinity co-assembly was executed using the following key parameters:

- `--seqType fq`: Specifies that input files are in FASTQ format.
- `--left` and `--right`: Combined paired-end reads from all samples.
- `--max_memory 200G`: Allocates up to 200 GB of RAM for assembly.
- `--CPU 8`: Uses 8 processing threads.
- `--normalize_max_read_cov 200`: Performs in silico read normalization to reduce redundancy and computational requirements while maintaining transcript diversity.
- `--output`: Defines the output directory.
- `--full_cleanup`: Removes intermediate files after completion to reduce disk space usage.
- `> "$log_file" 2>&1`: Redirects standard output and error messages to a log file for troubleshooting and reproducibility.

To improve performance and reduce I/O bottlenecks, Trinity was executed in the local `/scratch` directory of the compute node. The resulting assembly files were subsequently copied to permanent storage.

7.5.3 SLURM script used for Trinity co-assembly

We used a SLURM job to generate a global transcriptome assembly from all RNA-seq libraries.

SLURM script: `03_E_Trinity_Zamia_ALL_RNA.sh`

```
#!/bin/bash
#SBATCH -J TriAll_Zam
#SBATCH -o Tri_ALL_Zam_%j.out
#SBATCH -p large
#SBATCH --cpus-per-task=8
#SBATCH --mem=200G
#SBATCH --time=21-00:00:00
#SBATCH --mail-type=ALL
#SBATCH --mail-user=gfpeb@ulaval.ca
```

Job	Runtime	JobId	Partition	Cpu	Cpu(%)	Mem(Go)	Used
↪ Unused	Mem(%)						

```

---          +
↪          ↪
TriAll Zam 367h52m 323921 ibismini | 8 18.40% ** | 200.00 162.34
↪ 37.66 81.17%

```

```

#####
# 1. Activar entorno conda con Trinity
#####
source /prg/miniconda/3/etc/profile.d/conda.sh
conda activate /home/jcvil9/.conda/envs/trinity_env

#####
# 2. Definir rutas
#####
base_dir="/project/jcvaguilar/users/jcvil9/Gab_Zamia_Transcri"
input_base="${base_dir}/01_clean_reads_Zamia_RNA_trimmomatic"
output_base="${base_dir}/03_trinity_separated"

# Fecha y nombres
fecha=$(date +%Y%m%d)
scratch_dir="/scratch/${USER}/${SLURM_JOB_ID}/Zamia_all_${fecha}_trinity"
final_dir="${output_base}/Zamia_all_${fecha}_trinity"

# Crear carpetas
mkdir -p "$scratch_dir"
mkdir -p "$final_dir"

# Archivos concatenados
R1_all="${output_base}/all_samples_R1.fastq.gz"
R2_all="${output_base}/all_samples_R2.fastq.gz"

#####
# 3. Concatenar lecturas si no existen
#####
if [[ ! -f "$R1_all" || ! -f "$R2_all" ]]; then
    echo "Concatenando lecturas limpias..."
    cat ${input_base}/*/*_R1_paired.fastq.gz > "$R1_all"
    cat ${input_base}/*/*_R2_paired.fastq.gz > "$R2_all"

    # Guardar listado de muestras usadas
    ls ${input_base}/*/*_R1_paired.fastq.gz >
    ↪ "${output_base}/samples_concat_R1.txt"
    ls ${input_base}/*/*_R2_paired.fastq.gz >
    ↪ "${output_base}/samples_concat_R2.txt"
fi

#####
# 4. Ejecutar Trinity en /scratch

```

```
#####
log_file="${final_dir}/Trinity_all_${fecha}.log"

echo "Ejecutando Trinity conjunto en /scratch..."
Trinity \
  --seqType fq \
  --left "$R1_all" \
  --right "$R2_all" \
  --max_memory 200G \
  --CPU 8 \
  --normalize_max_read_cov 200 \
  --output "$scratch_dir" \
  --full_cleanup \
  > "$log_file" 2>&1

#####
# 5. Copiar resultados esenciales a /project
#####
echo "Copiando resultados de Trinity a $final_dir ..."
cp "$scratch_dir"/Trinity.fasta \
  "$scratch_dir"/Trinity.fasta.gene_trans_map \
  "$scratch_dir"/Trinity.timing \
  "$scratch_dir"/Trinity.stats \
  "$scratch_dir"/Trinity.fasta.seqLengths \
  "$scratch_dir"/recursive_trinity.cmds \
  "$scratch_dir"/*.log \
  "$final_dir/" 2>/dev/null || echo "Algunos archivos opcionales no se
↳ encontraron."

echo "Trinity co-assembly finalizado el $(date)"

conda deactivate
```

7.5.4 Trinity co-assembly output files

The output files were stored in a dedicated directory containing the global transcriptome assembly and associated metadata.

```
03_trinity_separated/
  Zamia_all_20250617_trinity/
    Trinity.fasta
    Trinity.fasta.gene_trans_map
    Trinity.stats
    Trinity.timing
    Trinity.fasta.seqLengths
    recursive_trinity.cmds
    Trinity_all_20250617.log
```

The resulting assembly represents a comprehensive reference transcriptome for *Zamia* and was used in all subsequent analyses, including assembly quality assessment, contamination filtering, taxonomic assignment, and functional annotation.

8 Assembly quality assessment with TransRate

TransRate is a reference-free tool that evaluates de novo transcriptome assemblies based on: - Read mapping rates, - Fragment coverage, - Potential chimeras or incomplete contigs.

It generates a set of statistics that help determine how well the reads support the assembled transcripts. This provides a **complementary perspective** to tools like TrinityStats or BUSCO.

For more information, see TransRate site.

8.1 SLURM script used for TransRate

The following script was used to all individual samples using a SLURM job array.

SLURM script: 04_TransRate_Zamia_RNA.sh

```
#!/bin/bash

#SBATCH -J TraRate_Zam
#SBATCH -o TransRate_Zam_test.out
#SBATCH -p small
#SBATCH --cpus-per-task=1
#SBATCH --array=6
#SBATCH --mem=8G
#SBATCH --time=1-00:00:00
#SBATCH --mail-type=ALL
#SBATCH --mail-user=gfpeb@ulaval.ca

#####

module load usearch/9.2.64_i86linux32
module load bowtie/2.3.4.1
module load eXpress/1.5.1
module load salmon/0.11.0
module load transrate/1.0.3

#####
# Definir rutas

base_dir="/project/jcvaguilar/users/jcvil9/Gab_Zamia_Transcri"
samples_list="${base_dir}/samples_list.txt"
reads_base="${base_dir}/01_clean_reads_Zamia_RNA_trimmomatic"
trinity_base="${base_dir}/03_trinity_separated"
```

```

results_base="${base_dir}/04_transrate_results"

#####
# Obtener nombre de muestra

sample=$(sed -n "${SLURM_ARRAY_TASK_ID}p" "$samples_list" | tr -d '[:space:]')
fecha=$(date +%Y%m%d)

#####
# Buscar ensamblaje más reciente

assembly_dir=$(find "$trinity_base" -maxdepth 1 -type d -name
↪ "${sample}_*_trinity" | sort | tail -n 1)
assembly_fasta=$(find "$assembly_dir" -maxdepth 1 -type f -name
↪ "${sample}_*_trinity.Trinity.fasta" | head -n 1)

#####
# Definir rutas de archivos

left_reads="${reads_base}/${sample}/${sample}_R1_paired.fastq.gz"
right_reads="${reads_base}/${sample}/${sample}_R2_paired.fastq.gz"
output_dir="${results_base}/${sample}_${fecha}_transrate"
log_file="${output_dir}/${sample}_${fecha}_transrate.log"

#####
# Crear directorio de salida

mkdir -p "$output_dir"

#####
# Ejecutar Transrate

transrate -a "$assembly_fasta" \
          --left "$left_reads" \
          --right "$right_reads" \
          -t 1 \
          --output "$output_dir" &> "$log_file"

```

8.2 Transrate output files per sample

The output files will be stored in a new folder named *04_transrate_results*, organized into subdirectories by sample, each containing the trimmed read files and associated logs.

The following statistics summarize the quality of the de novo transcriptome assembly (Trinity output) as assessed by TransRate:


```

...
[ INFO] 2025-06-25 15:20:54 : Calculating contig metrics...
[ INFO] 2025-06-25 15:21:37 : Contig metrics:
[ INFO] 2025-06-25 15:21:37 : -----
[ INFO] 2025-06-25 15:21:37 : n seqs                260669
[ INFO] 2025-06-25 15:21:37 : smallest                182
[ INFO] 2025-06-25 15:21:37 : largest                21955
[ INFO] 2025-06-25 15:21:37 : n bases              153009149
[ INFO] 2025-06-25 15:21:37 : mean len             586.97
[ INFO] 2025-06-25 15:21:37 : n under 200          26
[ INFO] 2025-06-25 15:21:37 : n over 1k           36339
[ INFO] 2025-06-25 15:21:37 : n over 10k           78
[ INFO] 2025-06-25 15:21:37 : n with orf          37212
[ INFO] 2025-06-25 15:21:37 : mean orf percent     56.19
[ INFO] 2025-06-25 15:21:37 : n90                  236
[ INFO] 2025-06-25 15:21:37 : n70                  363
[ INFO] 2025-06-25 15:21:37 : n50                 1265
[ INFO] 2025-06-25 15:21:37 : n30                 2338
[ INFO] 2025-06-25 15:21:37 : n10                 4185
[ INFO] 2025-06-25 15:21:37 : gc                   0.41
[ INFO] 2025-06-25 15:21:37 : bases n              0
[ INFO] 2025-06-25 15:21:37 : proportion n         0.0
[ INFO] 2025-06-25 15:21:37 : Contig metrics done in 43 seconds
...

```

The following metrics were extracted from the transrate.log output and provide a snapshot of assembly quality:

```

library(knitr)
library(tibble)

# Crear tabla con métricas e interpretaciones
transrate_metrics <- tribble(
  ~Metric, ~Value, ~Interpretation,
  "Number of contigs", "260,669", "Total assembled transcripts; reflects
  ↪ transcriptome complexity.",
  "Smallest contig", "182 bp", "Very short contigs; often discarded in downstream
  ↪ analysis.",
  "Largest contig", "21,955 bp", "Longest assembled sequence; may represent
  ↪ full-length gene.",
  "Total bases", "153,009,149", "Total number of assembled bases; reflects overall
  ↪ assembly size.",
  "Mean contig length", "586.97 bp", "Average transcript length; indicates
  ↪ assembly fragmentation or completeness.",
  "Contigs < 200 bp", "26", "Very short contigs; likely non-informative.",
  "Contigs > 1,000 bp", "36,339", "Putative full-length or near-complete
  ↪ transcripts.",

```

```

"Contigs > 10,000 bp", "78", "Very long transcripts; may indicate full-length
  ↪ genes or chimeras.",
"Contigs with ORFs", "37,212", "Transcripts with open reading frames; suggests
  ↪ coding potential.",
"Mean ORF %", "56.19%", "Average percentage of transcript covered by an ORF.",
"GC content", "41%", "Consistent with plant transcriptomes.",
"N50", "1,265 bp", "Half of the assembly is in contigs of this length or
  ↪ longer.",
"N90", "236 bp", "Length above which the top 90% of contigs fall.",
"N70", "363 bp", "Length above which the top 70% of contigs fall.",
"N30", "2,338 bp", "Length above which the top 30% of contigs fall.",
"N10", "4,185 bp", "Length above which the top 10% of contigs fall."
)

# Mostrar tabla
kable(transrate_metrics, caption = "Summary of Trinity assembly metrics and
  ↪ interpretations.")

```

Table 2: Summary of Trinity assembly metrics and interpretations.

Metric	Value	Interpretation
Number of contigs	260,669	Total assembled transcripts; reflects transcriptome complexity.
Smallest contig	182 bp	Very short contigs; often discarded in downstream analysis.
Largest contig	21,955 bp	Longest assembled sequence; may represent full-length gene.
Total bases	153,009,149	Total number of assembled bases; reflects overall assembly size.
Mean contig length	586.97 bp	Average transcript length; indicates assembly fragmentation or completeness.
Contigs < 200 bp	26	Very short contigs; likely non-informative.
Contigs > 1,000 bp	36,339	Putative full-length or near-complete transcripts.
Contigs > 10,000 bp	78	Very long transcripts; may indicate full-length genes or chimeras.
Contigs with ORFs	37,212	Transcripts with open reading frames; suggests coding potential.
Mean ORF %	56.19%	Average percentage of transcript covered by an ORF.
GC content	41%	Consistent with plant transcriptomes.
N50	1,265 bp	Half of the assembly is in contigs of this length or longer.
N90	236 bp	Length above which the top 90% of contigs fall.
N70	363 bp	Length above which the top 70% of contigs fall.
N30	2,338 bp	Length above which the top 30% of contigs fall.
N10	4,185 bp	Length above which the top 10% of contigs fall.

High ORF content (~56%) and a substantial number of **contigs >1,000 bp** suggest good

assembly completeness and biological relevance.

Low number of **short contigs** (<200 bp) indicates good filtering or normalization.

N50 = 1,265 bp is typical for de novo assemblies from short-read RNA-seq data.

GC content (41%) matches plant expectations and suggests no major contamination.

8.3 TransRate Assessment of the Global Trinity Assembly

In addition to the quality assessment performed for each individual transcriptome assembly, TransRate was also used to evaluate the global Trinity co-assembly generated from all RNA-seq libraries combined.

The analysis was conducted using the reference transcriptome obtained from the Trinity co-assembly and the concatenated paired-end reads used during assembly. This approach provides an overall assessment of assembly quality and read support across the complete dataset.

The assembly file was automatically identified from the most recent Trinity co-assembly directory:

```
03_trinity_separated/Zamia_all_YYYYMMDD_trinity/
```

The paired-end reads used for evaluation corresponded to the concatenated files:

- all_samples_R1.fastq.gz
- all_samples_R2.fastq.gz

The resulting TransRate output files were stored in:

```
04_transrate_results/Zamia_all_transrate
```

8.3.1 TransRate command parameters

The global assembly assessment was executed using the following key parameters:

- **-a**: Trinity co-assembly FASTA file used as the reference transcriptome.
- **--left** and **--right**: Concatenated paired-end reads from all samples.
- **-t 2**: Number of threads used by TransRate.
- **--output**: Directory where all quality assessment results were stored.
- **&> "\$log_file"**: Redirects standard output and error messages to a log file for reproducibility and troubleshooting.

Unlike the individual analyses, which evaluated each sample independently, this assessment measures the overall support of the complete RNA-seq dataset for the global transcriptome assembly.

8.3.2 SLURM script used for global TransRate assessment

We used a dedicated SLURM job to evaluate the Trinity co-assembly generated from all RNA-seq libraries.

SLURM script: 04_B_TransRate_Zamia_ALL_RNA.sh

```
#!/bin/bash

#SBATCH -J TraRa_Zam_All
#SBATCH -o TraRa_Zam_All.out
#SBATCH -p medium
#SBATCH --cpus-per-task=5
#SBATCH --mem=30G
#SBATCH --time=7-00:00:00
#SBATCH --mail-type=ALL
#SBATCH --mail-user=gfpeb@ulaval.ca

#####
# 1. Cargar módulos necesarios
#####

module load usearch/9.2.64_i86linux32
module load bowtie/2.3.4.1
module load eXpress/1.5.1
module load salmon/0.11.0
module load transrate/1.0.3

#####
# 2. Definir rutas
#####

base_dir="/project/jcvaguilar/users/jcvil9/Gab_Zamia_Transcri"
trinity_dir="${base_dir}/03_trinity_separated"
results_base="${base_dir}/04_transrate_results"

# Detectar ensamblaje conjunto automáticamente
assembly_dir=$(find "$trinity_dir" -maxdepth 1 -type d -name "Zamia_all*_trinity"
↪ | sort | tail -n 1)
assembly_fasta="${assembly_dir}/Trinity.fasta"

# Reads concatenados
left_reads="${trinity_dir}/all_samples_R1.fastq.gz"
right_reads="${trinity_dir}/all_samples_R2.fastq.gz"

# Salida Transrate
output_dir="${results_base}/Zamia_all_transrate"
log_file="${output_dir}/transrate_zamia_all.log"

# Crear directorio de salida
mkdir -p "$output_dir"

#####
```

```
# 3. Ejecutar Transrate
#####

transrate -a "$assembly_fasta" \
          --left "$left_reads" \
          --right "$right_reads" \
          -t 2 \
          --output "$output_dir" &> "$log_file"

echo "Transrate global finalizado el $(date)"
```

8.3.3 TransRate output files

The quality assessment results generated by TransRate were stored in a dedicated output directory.

```
04_transrate_results/
  Zamia_all_transrate/
    assemblies.csv
    contigs.csv
    transrate.log
    metrics.json
    reads_mapped.bam
    ...
```

The resulting metrics provide a global assessment of transcriptome quality and read support across all RNA-seq libraries and complement the individual assembly evaluations presented above.

9 BUSCO: Assembly Completeness Assessment

To assess the completeness of the assembled transcriptome, we used BUSCO (Benchmarking Universal Single-Copy Orthologs). BUSCO evaluates the presence of evolutionarily conserved genes expected to be found in a given lineage—in this case, Embryophyta (land plants). The tool classifies each ortholog into four categories:

- **Complete (C):** the ortholog is fully present in the assembly.
- **Single-copy (S):** appears only once, as expected.
- **Duplicated (D):** appears more than once; common in transcriptomes due to isoforms.
- **Fragmented (F):** only partial gene sequences were found.
- **Missing (M):** the ortholog was not detected.

For more information, see BUSCO site.

9.1 SLURM script used for BUSCO

Note: Before running BUSCO, it is necessary to download the appropriate ortholog database (e.g., `embryophyta_odb10`). We recommend storing it in a local directory for efficient and reproducible access, especially when working on shared computing environments or clusters.

BUSCO database.

The following script was used to all samples using a SLURM job array.

SLURM script: `05_A_BUSCO_Zamia_RNA.sh`

```
#!/bin/bash

#SBATCH -J BUSCO_Zam
#SBATCH -o BUSCO_Zamia_RNA_%j.out
#SBATCH -p medium
#SBATCH --array=6
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=2
#SBATCH --mem=15G
#SBATCH --time=2-00:00:00
#SBATCH --mail-type=ALL
#SBATCH --mail-user=gfpeb@ulaval.ca

#####
## activo python aunque Stephan me dice que no para ver qué pasa.
#module load python/3.7
module load ncbiblast/2.11.0
module load hmmer/3.3
module load bamtools/2.5.1
module load augustus/3.3.2
module load miniconda/3
module load busco/5.0.0

#####
# Activar entorno conda con BUSCO

eval "$(conda shell.bash hook)"
conda activate busco5

#####
# Rutas

base_dir="/project/jcvaguilar/users/jcvil9/Gab_Zamia_Transcri"
samples_list="${base_dir}/samples_list.txt"
trinity_base="${base_dir}/03_trinity_separated"
busco_base="${base_dir}/05_A_Busco_Zamia_RNA"
busco_data_dir="${base_dir}/busco_downloads"
```

```

#####
# Bases de datos a comparar (versiones ODB12 descargadas localmente)
# viridiplantae_odb10 - plantas verdes (algas + terrestres)
# embryophyta_odb10 - plantas terrestres
# eukaryota_odb10 - eucariotas generales
# eudicots_odb10 - solo angiospermas eudicotiledóneas.
# liliopsida_odb10 - solo monocotiledóneas.
dbs=("viridiplantae_odb10" "embryophyta_odb10" "eukaryota_odb10" "eudicots_odb10"
↪ "liliopsida_odb10")

dbs=("viridiplantae_odb10" "embryophyta_odb10")

#####
# Obtener muestra

sample=$(sed -n "${SLURM_ARRAY_TASK_ID}p" "$samples_list" | tr -d '[:space:]')
fecha=$(date +%Y%m%d)
trinity_dir=$(find "$trinity_base" -maxdepth 1 -type d -name "${sample}_*_trinity"
↪ | sort | tail -n 1)
fasta_file=$(find "$trinity_dir" -maxdepth 1 -type f -name "*.Trinity.fasta" |
↪ head -n 1)

#####
# Ejecutar BUSCO para cada base de datos

for db in "${dbs[@]"; do
    db_path=$(find "$busco_data_dir" -maxdepth 1 -type d -name "${db}*" | head -n 1)
    if [[ ! -d "$db_path" ]]; then
        echo "Base de datos BUSCO no encontrada: $db" >&2
        echo -e "${sample}\t${db}\tNA\tNA\tNA\tNA" >> "$summary_table"
        continue
    fi

    db_flag="-l $db_path"
    db_suffix=$(basename "$db_path")
    output_dir="${busco_base}/${sample}_${fecha}_busco_${db_suffix}"
    mkdir -p "$output_dir"
    log_file="${output_dir}/${sample}_${db_suffix}.busco.log"

    echo "Ejecutando BUSCO para $sample con DB: $db_suffix"
    echo "Salida BUSCO se guardará en: $log_file"

    busco -i "$fasta_file" \
        -o "${sample}_${db_suffix}" \
        -m transcriptome \
        $db_flag \
        --cpu 1 \
        --out_path "$output_dir" \

```

```

    > "$log_file" 2>&1

    echo "BUSCO finalizado para $sample con DB: $db_suffix"

done
conda deactivate

```

9.2 BUSCO output files per sample

The output files will be stored in a new folder named *05_A_Busco_Zamia_RNA*, organized into subdirectories by sample, each containing the BUSCO files and associated logs.

```

# BUSCO version is: 5.0.0
# The lineage dataset is: embryophyta_odb10 (Creation date: 2024-01-08, number of
  ↳ species: 50, number of BUSCOs: 1614)
# Summarized benchmarking in BUSCO notation for file
  ↳ /project/jcvaguilar/users/jcvil9/Gab_Zamia_Transcri/03_trinity_separated/JCV23-0101_2025061
# BUSCO was run in mode: transcriptome

***** Results: *****

C:90.2%[S:50.6%,D:39.6%],F:3.0%,M:6.8%,n:1614
1456   Complete BUSCOs (C)
817   Complete and single-copy BUSCOs (S)
639   Complete and duplicated BUSCOs (D)
48    Fragmented BUSCOs (F)
110   Missing BUSCOs (M)
1614   Total BUSCO groups searched

```

9.3 BUSCO summary

The following SLURM summarizes the results of the BUSCO step for all samples.

SLURM script: 05_B_BUSCO_resumen_Zamia_RNA.sh

```

#!/bin/bash

#SBATCH -J BUSCO_Zam_Sum
#SBATCH -o BUSCO_Zamia_RNA_Sum_%j.out
#SBATCH -e BUSCO_Zamia_RNA_Sum_%A_%a.err
#SBATCH -p small
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=2G
#SBATCH --time=1-00:00:00
#SBATCH --mail-type=ALL

```



```

#SBATCH --mail-user=gfpeb@ulaval.ca

#####
module load miniconda/3

#####
# Activar entorno conda con BUSCO

eval "$(conda shell.bash hook)"
conda activate busco5

#####
# Ruta base de resultados BUSCO

busco_base="/project/jcvaguilar/users/jcvil9/Gab_Zamia_Transcri/05_A_Busco_Zamia_RNA"

# Obtener fecha actual
fecha=$(date +%Y%m%d)

# Nombre del archivo de salida con fecha
output_file="${busco_base}/busco_multi_summary_${fecha}.tsv"

# Crear encabezado del resumen
echo -e "Sample\tDatabase\tComplete\tSingle\tDuplicated\tFragmented\tMissing" >
  ↪ "$output_file"

#####
# Buscar todos los archivos short_summary*.txt y extraer info
find "$busco_base" -type f -name "short_summary*.txt" | while read summary_file;
  ↪ do
    filename=$(basename "$summary_file")

    # Extraer base de datos y muestra
    db=$(echo "$filename" | cut -d '.' -f3)
    sample_full=$(echo "$filename" | cut -d '.' -f4)
    sample=$(echo "$sample_full" | sed "s/_${db}/")

    # Extraer línea de resumen con los porcentajes
    summary_line=$(grep "C:" "$summary_file" | head -n 1 | \
      sed
      ↪ 's/.*C:\([0-9.]*\)%\[S:\([0-9.]*\)%\[D:\([0-9.]*\)%\[F:\([0-9.]*\)%\[M:\([0-9.]*\)%\[

    # Verificar que los campos sean válidos
    if [[ -n "$sample" && -n "$db" && -n "$summary_line" ]]; then
      echo -e "${sample}\t${db}\t${summary_line}" >> "$output_file"
    fi
  done

```

```
echo "Tabla resumen generada: $output_file"
conda deactivate
```

```
library(readr)
library(dplyr)
library(knitr)

# Leer el archivo CSV desde tu computadora
Busco_summary <- read_tsv(file.path(FIG, "busco_multi_summary_20250701.tsv"))

# Mostrar tabla ordenada por muestra
Busco_summary <- Busco_summary %>% arrange(Sample)

# Mostrar tabla
kable(Busco_summary, caption = "Busco summary for each sample")
```

Table 3: Busco summary for each sample

Sample	Database	Complete	Single	Duplicated	Fragmented	Missing
JCV23-0101	viridiplantae_odb10	97.2	43.3	53.9	2.1	0.7
JCV23-0101	embryophyta_odb10	90.2	50.6	39.6	3.0	6.8
JCV23-0102	viridiplantae_odb10	96.9	40.7	56.2	2.4	0.7
JCV23-0102	embryophyta_odb10	89.6	50.6	39.0	3.9	6.5
JCV23-0201	viridiplantae_odb10	96.7	44.7	52.0	2.6	0.7
JCV23-0201	embryophyta_odb10	90.6	55.9	34.7	3.3	6.1
JCV23-0202	viridiplantae_odb10	95.8	37.4	58.4	3.5	0.7
JCV23-0202	embryophyta_odb10	87.1	47.3	39.8	5.0	7.9
JCV23-0301	viridiplantae_odb10	94.1	40.5	53.6	4.7	1.2
JCV23-0301	embryophyta_odb10	84.2	49.6	34.6	6.4	9.4
JCV23-0302	viridiplantae_odb10	98.6	41.4	57.2	0.5	0.9
JCV23-0302	embryophyta_odb10	93.3	51.1	42.2	2.0	4.7
JCV23-0401	viridiplantae_odb10	92.5	45.4	47.1	6.4	1.1

Sample	Database	Complete	Single	Duplicated	Fragmented	Missing
JCV23-0401	embryophyta_odb10	83.2	50.2	33.0	6.1	10.7
JCV23-0402	viridiplantae_odb10	95.3	44.5	50.8	3.8	0.9
JCV23-0402	embryophyta_odb10	87.5	51.3	36.2	4.9	7.6
JCV23-0501	viridiplantae_odb10	86.8	45.9	40.9	9.6	3.6
JCV23-0501	embryophyta_odb10	74.0	49.4	24.6	11.0	15.0
JCV23-0502	viridiplantae_odb10	89.9	40.0	49.9	8.0	2.1
JCV23-0502	embryophyta_odb10	80.7	48.6	32.1	7.3	12.0
JCV23-0601	viridiplantae_odb10	77.2	46.1	31.1	16.7	6.1
JCV23-0601	embryophyta_odb10	63.8	42.9	20.9	13.4	22.8
JCV23-0602	viridiplantae_odb10	95.5	42.8	52.7	3.8	0.7
JCV23-0602	embryophyta_odb10	87.4	51.9	35.5	4.6	8.0

```

library(readr)
library(dplyr)
library(tidyr)
library(ggplot2)

# Leer datos
Busco_summary <- read_tsv(file.path(FIG, "busco_multi_summary_20250701.tsv"))

# Crear columna combinada de muestra y base de datos
busco <- Busco_summary %>%
  mutate(Sample_DB = paste(Sample, Database, sep = " | ")) %>%
  select(Sample, Sample_DB, Single, Duplicated, Fragmented, Missing)

# Ordenar muestras alfabéticamente
busco <- busco %>% arrange(Sample)

# Convertir a formato largo
busco_long <- busco %>%
  pivot_longer(cols = c(Single, Duplicated, Fragmented, Missing),
               names_to = "Category", values_to = "Percent")

# Definir orden de categorías y muestras
busco_long$Category <- factor(busco_long$Category,

```

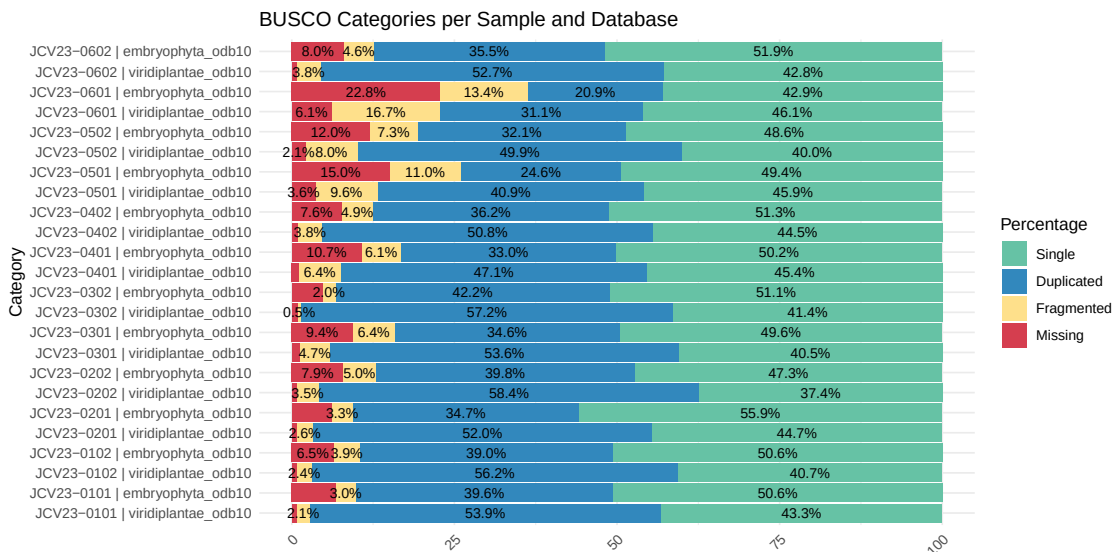
```

        levels = c("Single", "Duplicated", "Fragmented",
                  ↪ "Missing"))
busco_long$Sample_DB <- factor(busco_long$Sample_DB,
                              levels = unique(busco$Sample_DB))

# Colores personalizados
busco_colors <- c(
  "Single" = "#66c2a5",
  "Duplicated" = "#3288bd",
  "Fragmented" = "#fee08b",
  "Missing" = "#d53e4f"
)

# Crear gráfico
ggplot(busco_long, aes(x = Percent, y = Sample_DB, fill = Category)) +
  geom_bar(stat = "identity") +
  geom_text(aes(label = sprintf("%.1f%%", Percent)),
            position = position_stack(vjust = 0.5),
            size = 3, color = "black", check_overlap = TRUE) +
  scale_fill_manual(values = busco_colors) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  labs(
    title = "BUSCO Categories per Sample and Database",
    x = NULL,
    y = "Category",
    fill = "Percentage"
  )

```



BUSCO categories and reference values

Single-copy (Single): - Represents the percentage of complete BUSCO genes that were found as single copies. - Reference: Values above **40–50%** are generally acceptable, especially in transcriptomes; higher values indicate less duplication and better gene model representation.

Duplicated: - Indicates complete genes that appear more than once. - Reference: A moderate percentage (**e.g., 30–60%**) is common in transcriptomes due to isoforms or gene families. Very high duplication (**>70%**) might suggest assembly artifacts.

Fragmented: - BUSCO genes detected but not in full length, indicating partial or incomplete gene reconstruction. - Reference: Should be kept below **10%** for good assemblies; higher values may point to fragmentation or low expression levels.

Missing: - BUSCO genes not found in the assembly. - Reference: Ideally below **5–10%**. Higher values suggest incomplete transcriptome coverage, insufficient sequencing depth, or taxonomic mismatch with the BUSCO database

10 Sequence Similarity Searches: BLASTx and BLASTn

To identify potential homologous sequences and obtain taxonomic information for the assembled transcripts, sequence similarity searches were performed using both BLASTx and BLASTn. These analyses provide complementary information by comparing the assembled transcripts against protein and nucleotide databases, respectively.

BLAST results were subsequently used for taxonomic assignment, contamination assessment, and downstream filtering steps.

10.1 BLASTx: Similarity Search Against the NR Protein Database

BLASTx translates nucleotide sequences into all six possible reading frames and compares the resulting protein sequences against a protein database. This approach is particularly useful for transcriptome analyses because protein sequences are generally more conserved than nucleotide sequences, allowing the detection of distant homologs and functional annotation of assembled transcripts.

For this study, Trinity transcript assemblies were searched against the NCBI non-redundant protein database (NR release 115).

The analysis was performed on the Trinity assemblies located in:

03_trinity_separated/

The output files were stored in:

06_BlastX_Zamia_RNA/

10.1.1 BLASTx command parameters

The BLASTx searches were executed using the following key parameters:

- `-db nr.115`: NCBI non-redundant protein database.
- `-max_target_seqs 10`: Retains up to 10 hits per query sequence.

- `-best_hit_score_edge 0.05`: Filters suboptimal hits based on score proximity.
- `-best_hit_overhang 0.25`: Controls the best-hit filtering algorithm.
- `-max_hsps 1`: Retains only the best alignment segment per subject sequence.
- `-evalue 1e-25`: Stringent significance threshold.
- `-num_threads`: Number of CPU threads allocated by SLURM.
- `-outfmt 6`: Tabular output containing taxonomic and alignment information.

These parameters were selected to maximize annotation reliability while reducing the number of low-confidence matches.

10.1.2 SLURM script used for BLASTx analysis

We used a SLURM array job to perform BLASTx searches independently for each Trinity assembly.

SLURM script: 06_A_BlastxTrinity_proteins_Zamia_RNA.sh

```
#!/bin/bash
#SBATCH -J blastx_Zamia.s
#SBATCH -o blastxnr115-%j.out
#SBATCH -p large
#SBATCH --array=6
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=14
#SBATCH --mem=50G
#SBATCH --time=7-00:00:00
#SBATCH --mail-type=ALL
#SBATCH --mail-user=gfpeb@ulaval.ca

#####
##### Job          Runtime  JobId      Partition / Cpu  Cpu(%) /
↳ Mem(Go)  Used  Unused  Mem(%)
##### ---          -
↳ -----  ----  -----  -----
##### blastx Zamia.s 40h15m  311385 9  ibismini  / 14  96.39% / 200.00
↳ 0.10 199.90 0.05% ****

#####

module load ncbiblast/2.16.0

##### Rutas generales

base_dir="/project/jcvaguilar/users/jcvil9/Gab_Zamia_Transcri"
samples_list="${base_dir}/samples_list.txt"
trinity_base="${base_dir}/03_trinity_separated"
blastx_base="${base_dir}/06_BlastX_Zamia_RNA"
```

```

taxdump_dir="${base_dir}/new_taxdump"

##### Exporta la ruta donde está tu NR, base de datos taxonomicos

export BLASTDB=/biodata/blast

##### Obtener muestra

sample=$(sed -n "${SLURM_ARRAY_TASK_ID}p" "$samples_list" | tr -d '[:space:]')
fecha=$(date +%Y%m%d)
trinity_dir=$(find "$trinity_base" -maxdepth 1 -type d -name "${sample}_*_trinity"
↪ | sort | tail -n 1)
fasta_file=$(find "$trinity_dir" -maxdepth 1 -type f -name "*.Trinity.fasta" |
↪ head -n 1)

##### Crear carpeta de salida

output_dir="${blastx_base}/${sample}_${fecha}_blastx_nr115"
mkdir -p "$output_dir"
log_file="${output_dir}/${sample}_blastx_nr115.log"

##### Ejecutar BLASTx contra NR.115

blastx -query "$fasta_file" \
  -db nr.115 \
  -max_target_seqs 10 \
  -best_hit_score_edge 0.05 \
  -best_hit_overhang 0.25 \
  -max_hsps 1 \
  -evaluate 1e-25 \
  -num_threads "$SLURM_CPUS_PER_TASK" \
  -outfmt "6 \
    qseqid \
    staxids \
    bitscore \
    sseqid \
    sscinames \
    pident \
    length \
    mismatch \
    gapopen \
    qstart \
    qend \
    sstart \
    send \
    evaluate" \
  -out "${output_dir}/${sample}_blastx.outfmt6" \
  >> "$log_file" 2>&1

```

10.1.3 BLASTx output files

```
06_BlastX_Zamia_RNA/  
  JCV23-0101_20250617_blastx_nr115/  
    JCV23-0101_blastx.outfmt6  
    JCV23-0101_blastx_nr115.log  
  JCV23-0102_20250617_blastx_nr115/  
  JCV23-0201_20250617_blastx_nr115/  
  ...
```

The BLASTx output contains taxonomic identifiers, protein matches, alignment statistics, and sequence similarity metrics that are later used for taxonomic classification and contamination filtering.

10.2 BLASTn: Similarity Search Against the NT Database

BLASTn compares nucleotide sequences directly against a nucleotide database. In transcriptome analyses, BLASTn complements BLASTx by identifying highly similar nucleotide sequences and providing additional taxonomic evidence, particularly for transcripts lacking protein-level matches.

For this study, Trinity transcript assemblies were searched against the NCBI nucleotide database (NT).

The analysis was performed on the Trinity assemblies located in:

```
03_trinity_separated/
```

The output files were stored in:

```
07_BLASTn_Zamia_RNA/
```

10.2.1 BLASTn command parameters

The BLASTn searches were executed using the following key parameters:

- `-db nt`: NCBI nucleotide database.
- `-max_target_seqs 10`: Retains up to 10 hits per query sequence.
- `-best_hit_score_edge 0.05`: Best-hit filtering threshold.
- `-best_hit_overhang 0.25`: Controls the best-hit selection algorithm.
- `-max_hsps 1`: Retains only the best alignment segment.
- `-evalue 1e-25`: Stringent significance threshold.
- `-num_threads`: Number of CPU threads allocated by SLURM.
- `-outfmt 6`: Tabular output format including taxonomic information and alignment statistics.

These settings prioritize highly confident matches while maintaining computational efficiency.

10.2.2 SLURM script used for BLASTn analysis

We used a SLURM array job to perform BLASTn searches independently for each Trinity assembly.

SLURM script: 07_blastn.sh

```
#!/bin/bash

#SBATCH -J Blastn
#SBATCH -o Blastn-%j.out
#SBATCH -p medium
#SBATCH --array=6
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=2
#SBATCH --mem=20G
#SBATCH --time=5-00:00:00
#SBATCH --mail-type=ALL
#SBATCH --mail-user=gfpeb@ulaval.ca

#module load ncbiblast/2.9.0
module load ncbiblast/2.16.0

#Path to nt database
db="/biodata/blastdb_new/nt"

#####
# Rutas base

base_dir="/project/jcvaguilar/users/jcvil9/Gab_Zamia_Transcri"
samples_list="${base_dir}/samples_list.txt"
trinity_base="${base_dir}/03_trinity_separated"
blast_base="${base_dir}/07_BLASTn_Zamia_RNA"

# es necesario confirmar la ruta con el archivo nt en el servidor
# Base de datos BLAST

db="/biodata/blast/nt"

#####
# Obtener nombre de muestra

sample=$(sed -n "${SLURM_ARRAY_TASK_ID}p" "$samples_list" | tr -d '[:space:]')
fecha=$(date +%Y%m%d)

#####
# Buscar el archivo .Trinity.fasta correspondiente

trinity_dir=$(find "$trinity_base" -maxdepth 1 -type d -name "${sample}_*_trinity"
↪ | sort | tail -n 1)
```

```

fasta_file=$(find "$trinity_dir" -maxdepth 1 -type f -name "*.Trinity.fasta" |
↪ head -n 1)

#####
# Directorio y archivo de salida

output_dir="${blast_base}/${sample}_${fecha}_blastn"
mkdir -p "$output_dir"
log_file="${output_dir}/${sample}_blastn_${fecha}.log"

#####
# Ejecutar BLASTn

blastn -db "$db" \
    -query "$fasta_file" \
    -outfmt "6 qseqid sseqid staxids sscinames pident length mismatch gapopen
↪ qstart qend sstart send evalue bitscore" \
    -max_target_seqs 10 \
    -best_hit_score_edge 0.05 \
    -best_hit_overhang 0.25 \
    -max_hsps 1 \
    -evalue 1e-25 \
    -num_threads "$SLURM_CPUS_PER_TASK" \
    -out "${output_dir}/${sample}_blastn.tsv" >> "$log_file" 2>&1

echo "BLASTn terminado para $sample" >> "$log_file"

```

10.2.3 BLASTn output files

```

07_BLASTn_Zamia_RNA/
  JCV23-0101_20250617_blastn/
    JCV23-0101_blastn.tsv
    JCV23-0101_blastn_20250617.log
  JCV23-0102_20250617_blastn/
  JCV23-0201_20250617_blastn/
  ...

```

The BLASTn results provide complementary nucleotide-level evidence that can be integrated with BLASTx results to improve taxonomic assignment and identify potential contaminant sequences during downstream filtering procedures.

11 Minimap2: Mapping assembled transcripts

Minimap2 is a fast and accurate aligner designed for mapping long or short sequences to a reference genome or assembly. In transcriptome projects, it is commonly used to:

- Assess transcriptome completeness, by aligning assembled transcripts to a genome or reference sequence.
- Detect possible contamination, if transcripts align to unexpected taxa.
- Support structural annotation, by mapping to well-characterized genomes.
- Generate alignment files (e.g., BAM), which can be used for downstream analyses such as BlobToolKit or coverage estimation.

In this project, Minimap2 was used to align assembled transcripts to a reference genome in order to evaluate overall coverage and genomic distribution of the assembled data. The alignment results are also used as input for visualization tools and contamination assessment (e.g., BlobTools2).

For more information, see Minimap2 site.

11.1 Minimap2 command parameters

In this analysis, we used the following command:

```
minimap2 -ax sr --MD reference.fasta left_reads.fq right_reads.fq | \
samtools sort -@2 -O BAM -o output.bam
```

- **-ax sr:** This preset is used for short paired-end reads (e.g., Illumina). It activates spliced alignment with appropriate scoring and filtering for short-read RNA data.
- **--MD:** Instructs minimap2 to add MD tags to the alignment, which record mismatched positions. This is useful for downstream tools that inspect mismatch patterns or calculate coverage and identity.
- **reference.fasta:** Reference genome or transcriptome to which the reads were aligned. In this case, it corresponds to the assembled transcriptome or a reference genome of the closest species.
- **left_reads.fq** and **right_reads.fq:** Forward and reverse trimmed paired-end reads used for alignment.
- **[-@2]:** Sorts the resulting alignments using 2 threads for parallelism ([-@2]), optimizing speed on limited CPU systems.
- **-O BAM:** Specifies BAM format (binary version of SAM) as the output.
- **-o output.bam:** Names the final sorted BAM file.

This approach produces clean, indexed BAM files suitable for quality evaluation, coverage statistics, and integration with tools like BlobToolKit.

11.2 SLURM script used for Minimap2

SLURM script: 08_Minimap2_cov.sh

```
#!/bin/bash

#SBATCH -J Minim_Zam
#SBATCH -o Minim_Zamia_RNA_%j.out
```

```

#SBATCH -p medium
#SBATCH --array=6
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=10G
#SBATCH --time=2-00:00:00
#SBATCH --mail-type=ALL
#SBATCH --mail-user=gfpeb@ulaval.ca

#####

module load minimap2/2.28
module load samtools/1.21

#####
# Rutas base

base_dir="/project/jcvaguilar/users/jcvil9/Gab_Zamia_Transcri"
samples_list="${base_dir}/samples_list.txt"
trinity_base="${base_dir}/03_trinity_separated"
reads_base="${base_dir}/01_clean_reads_Zamia_RNA_trimmomatic"
output_base="${base_dir}/08_minimap2_bams"

#####
# Obtener nombre de muestra

sample=$(sed -n "${SLURM_ARRAY_TASK_ID}p" "$samples_list" | tr -d '[:space:]')
fecha=$(date +%Y%m%d)

#####
# Rutas de entrada

trinity_dir=$(find "$trinity_base" -maxdepth 1 -type d -name "${sample}_*_trinity"
↪ | sort | tail -n 1)
fasta_file=$(find "$trinity_dir" -maxdepth 1 -type f -name "*.Trinity.fasta" |
↪ head -n 1)

left_reads="${reads_base}/${sample}/${sample}_R1_paired.fastq.gz"
right_reads="${reads_base}/${sample}/${sample}_R2_paired.fastq.gz"

#####
# Directorio de salida

output_dir="${output_base}/${sample}_${fecha}_minimap2"
mkdir -p "$output_dir"
bam_file="${output_dir}/${sample}_assembly.bam"
log_file="${output_dir}/${sample}_minimap2_${fecha}.log"

```

```
#####
# Ejecutar alineamiento

echo "Iniciando alineamiento para ${sample} el ${fecha}" | tee "$log_file"

minimap2 -ax sr --MD "$fasta_file" "$left_reads" "$right_reads" 2>>"$log_file" | \
samtools sort -@2 -O BAM -o "$bam_file" 2>>"$log_file"

echo "Alineamiento completado para ${sample} el ${fecha}" | tee -a "$log_file"

#Use interactive mode and blobtools add --cov assembly.bam pour ajouter l'info au
↪ BlobDir
```

11.3 Minimap2 output files per sample

The output files will be stored in a new folder named *08_minimap2_bams*, organized into subdirectories by sample, each containing the Minimap2 files and associated logs.

The alignment process completed successfully for all samples, and no errors or interruptions were reported in the log files.

```
Iniciando alineamiento para JCV23-0101 el 20250625
[M::mm_idx_gen::3.316*1.42] collected minimizers
[M::mm_idx_gen::4.286*1.55] sorted minimizers
[M::main::4.444*1.50] loaded/built the index for 260669 target sequence(s)
[M::mm_mapopt_update::4.444*1.50] mid_occ = 1000
[M::mm_idx_stat] kmer size: 21; skip: 11; is_hpc: 0; #seq: 260669
[M::mm_idx_stat::4.689*1.47] distinct minimizers: 15037121 (70.56% are
↪ singletons); average occurrences: 1.625; average spacing: 6.263; total length:
↪ 153009149
[M::worker_pipeline::20.510*1.80] mapped 333642 sequences

[M::worker_pipeline::2434.531*1.75] mapped 333568 sequences
[M::worker_pipeline::2435.246*1.75] mapped 282054 sequences
[M::main] Version: 2.28-r1209
[M::main] CMD: minimap2 -ax sr --MD
[M::main] Real time: 2435.312 sec; CPU: 4264.802 sec; Peak RSS: 1.905 GB
[bam_sort_core] merging from 16 files and 2 in-memory blocks...
Alineamiento completado para JCV23-0101 el 20250625
```

The resulting BAM files will be used in subsequent analysis steps, including coverage estimation and contamination filtering with BlobToolKit.

Nome	Tamanho
JCV23-0101_assembly.bam	4.502.706 ...
JCV23-0101_minimap2_20250625.log	12 KB

Figure 2: Example of the output structure generated by Minimap2.

12 BlobTools2: Taxonomic and Coverage-Based Assessment of Transcriptome Assemblies

BlobTools2 is a framework designed to integrate sequence composition, read coverage, taxonomic assignments, and completeness metrics into a unified dataset. It facilitates the identification of contaminants, assessment of assembly quality, and visualization of taxonomic patterns within genome and transcriptome assemblies.

For this study, BlobTools2 was used to combine information from Trinity assemblies, BUSCO assessments, read coverage estimates generated with Minimap2, and taxonomic assignments obtained from BLAST analyses.

Additional information about BlobTools2 can be found at:

- <https://blobtoolkit.genomehubs.org/blobtools2/>
- <https://github.com/genomehubs/blobtoolkit>

The BlobTools2 workflow was performed in three major steps:

1. Creation of the BlobDir database.
2. Integration of BUSCO completeness information.
3. Addition of read coverage information from Minimap2 alignments.

12.1 BlobTools2 Database Creation

The first step consisted of creating a BlobTools2 database (BlobDir) using the Trinity assembly generated for each sample.

The analysis was performed using the transcriptome assemblies located in:

03_trinity_separated/

The resulting BlobDir databases were stored in:

09_BlobTools2_Zamia_RNA/

12.1.1 BlobTools2 create parameters

The database creation was executed using the following parameters:

- `--fasta`: Trinity assembly used as the reference dataset.
- `--taxid 33090`: Assigns Viridiplantae as the root taxonomic identifier.

- **create:** Generates a BlobDir database containing sequence information and metadata.

The resulting BlobDir serves as the central repository to which coverage, BUSCO, and taxonomic information are subsequently added.

12.1.2 SLURM script used for BlobDir creation

SLURM script: 09_Blob_create.sh

```
#!/bin/bash

#SBATCH -J Blob_Zamia
#SBATCH -o Blob_Zamia_%j.out
#SBATCH -c 1
#SBATCH -p small
#SBATCH --array=1
#SBATCH --time=1-00:00:00
#SBATCH --mail-type=ALL
#SBATCH --mail-user=gfpeb@ulaval.ca
#SBATCH --mem=8G

module load miniconda/3
module load blobtools/2

#Activate conda environment before running
eval "$(conda shell.bash hook)"
conda activate btk_env

#####
# Rutas base

base_dir="/project/jcvaguilar/users/jcvil9/Gab_Zamia_Transcri"
samples_list="${base_dir}/samples_list.txt"
trinity_base="${base_dir}/03_trinity_separated"
blob_base="${base_dir}/09_BlobTools2_Zamia_RNA"

#####
# Obtener nombre de muestra

sample=$(sed -n "${SLURM_ARRAY_TASK_ID}p" "$samples_list" | tr -d '[:space:]')
fecha=$(date +%Y%m%d)

#####
# Buscar el archivo .Trinity.fasta correspondiente

trinity_dir=$(find "$trinity_base" -maxdepth 1 -type d -name "${sample}_*_trinity"
↵ | sort | tail -n 1)
```

```

fasta_file=$(find "$trinity_dir" -maxdepth 1 -type f -name "*.Trinity.fasta" |
↪ head -n 1)

#####
# Directorio y archivo de salida

output_dir="${blob_base}/${sample}_${fecha}_blobdir"
mkdir -p "$output_dir"
log_file="${output_dir}/${sample}_blobtools_${fecha}.log"

#####
# Iniciar log

{
echo "==== BlobTools2 analysis for sample: $sample ====="
echo "Fecha: $(date)"
echo "FASTA: $fasta_file"
echo "Output: $output_dir"
echo "-----"
} > "$log_file"

#####
# Crear base de datos BlobTools (solo fasta)
# --taxid 33090 \ para Viridiplantae

/prg/blobtools/2/blobtools2/blobtools create \
  --fasta "$fasta_file" \
  --taxid 33090 \
  "$output_dir" \
  >> "$log_file" 2>&1
conda deactivate

```

12.1.3 BlobDir output files

```

09_BlobTools2_Zamia_RNA/
  JCV23-0101_20250617_blobdir/
    meta.json
    identifiers.json
    summary.json
    views/
    ...
  JCV23-0102_20250617_blobdir/
  JCV23-0201_20250617_blobdir/
  ...

```

The BlobDir contains the sequence metadata and serves as the foundation for all subsequent

BlobTools2 analyses.

12.2 Integration of BUSCO Completeness Metrics

To facilitate completeness-based filtering and visualization, BUSCO results were incorporated into each BlobDir database.

Because BUSCO reports included coordinate suffixes in transcript identifiers (e.g., TRINITY_DN6269_c0_g1_i12:76), a preprocessing step was required to remove the coordinate information and ensure consistency with Trinity sequence identifiers.

The BUSCO analyses used in this study were:

- `embryophyta_odb10`
- `viridiplantae_odb10`

Input files were obtained from:

`05_A_Busco_Zamia_RNA/`

Cleaned BUSCO tables were stored in:

`05_B_cleaned_busco/`

The updated BlobDir databases remained in:

`09_BlobTools2_Zamia_RNA/`

12.2.1 BlobTools2 BUSCO integration parameters

The BUSCO integration was performed using:

- `--busco`: BUSCO `full_table.tsv` file.
- `--replace`: Replaces any previous BUSCO dataset already present in the BlobDir.
- `add`: Adds BUSCO information to the existing BlobDir database.

12.2.2 SLURM script used for BUSCO integration

SLURM script: `10_A_Blob_add_Busco.sh`

```
#!/bin/bash

#SBATCH -J Blob-busco_cle-Zamia
#SBATCH -o Blob_add_busco_%j.out
#SBATCH -p small
#SBATCH --array=2-12
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=8G
```

```

#SBATCH --time=1-00:00:00
#SBATCH --mail-type=ALL
#SBATCH --mail-user=gfpeb@ulaval.ca

#####
# se ejecuto este codigo porque no estaban concordando los nombres entre el
↪ archivo .fasta de trinity y los resultados de Busco
# TRINITY_DN6269_c0_g1_i12 contra TRINITY_DN6269_c0_g1_i12:763-7038, se elimina
↪ :763-7038 para que concuerden los datos.
#####

module load miniconda/3
module load blobtools/2
module load python/3.11

#Activate conda environment before running
eval "$(conda shell.bash hook)"
conda activate btk_env

#####
# Rutas base

base_dir="/project/jcvaguilar/users/jcvil9/Gab_Zamia_Transcri"
samples_list="${base_dir}/samples_list.txt"
blobdir_base="${base_dir}/09_BlobTools2_Zamia_RNA"
busco_base="${base_dir}/05_A_Busco_Zamia_RNA"
clean_busco_dir="${base_dir}/05_B_cleaned_busco"

mkdir -p "$clean_busco_dir"

#####
# Obtener muestra seg n SLURM_ARRAY_TASK_ID

sample=$(sed -n "${SLURM_ARRAY_TASK_ID}p" "$samples_list" | tr -d '[:space:]')
echo "Procesando muestra: $sample"

blobdir=$(find "$blobdir_base" -maxdepth 1 -type d -name "${sample}_*_blobdir" |
↪ sort | tail -n 1)
if [[ ! -d "$blobdir" ]]; then
    echo "BlobDir no encontrado para $sample"
    exit 1
fi

log_file="${blobdir}/${sample}_add_busco_cleaned.log"
{
    echo "==== Limpieza + Integraci n BUSCO para muestra: $sample ====="
    echo "Fecha: $(date)"
}

```

```

echo "BlobDir: $blobdir"
echo "-----"
} > "$log_file"

#####
# Funci n de limpieza (conserva todas las columnas, limpia solo Sequence)

limpiar_busco() {
    local original="$1" cleaned="$2"
    {
        # 1) Versi n completa
        grep -m1 "^# BUSCO version" "$original"
        # 2) Dataset completo (con par ntesis intactos)
        grep -m1 "^# The lineage dataset" "$original"
        # 3) Encabezado completo de BUSCO (sin # para que sea columna)
        grep -m1 "^# Busco id" "$original"
        # 4) Datos reales, limpiando solo el sufijo :start-end en la columna 3
        awk -F'\t' 'BEGIN { OFS="\t" }
            !/^#/ && NF >= 3 {
                gsub(/:./, "", $3)
                print
            }' "$original"
    } > "$cleaned"
}

#####
# BUSCO embryophyta

busco_embryo_raw=$(find "$busco_base" -type f -path
↪  "$*${sample}*_busco_embryophyta_odb10*" -name "full_table.tsv" | head -n 1)
busco_embryo_clean="${clean_busco_dir}/${sample}_embryophyta_clean.tsv"

if [[ -f "$busco_embryo_raw" ]]; then
    echo "Limpieza y carga BUSCO embryophyta para $sample" | tee -a "$log_file"
    limpiar_busco "$busco_embryo_raw" "$busco_embryo_clean"

/prg/blobtools/2/blobtools2/blobtools add \
    --replace \
    --busco "$busco_embryo_clean" \
    "$blobdir" >> "$log_file" 2>&1

else
    echo " No se encontr BUSCO embryophyta para $sample" | tee -a "$log_file"
fi

#####
# BUSCO viridiplantae

```

```

busco_viridi_raw=$(find "$busco_base" -type f -path
↪  "*${sample}*_busco_viridiplantae_odb10*" -name "full_table.tsv" | head -n 1)
busco_viridi_clean="${clean_busco_dir}/${sample}_viridiplantae_clean.tsv"

if [[ -f "$busco_viridi_raw" ]]; then
    echo "Limpieza y carga BUSCO viridiplantae para $sample" | tee -a "$log_file"
    limpiar_busco "$busco_viridi_raw" "$busco_viridi_clean"

    /prg/blobtools/2/blobtools2/blobtools add \
    --replace \
    --busco "$busco_viridi_clean" \
    "$blobdir" >> "$log_file" 2>&1
else
    echo " No se encontr BUSCO viridiplantae para $sample" | tee -a "$log_file"
fi

echo " Finalizado: $sample" | tee -a "$log_file"
echo "-----" >> "$log_file"

conda deactivate

```

12.2.3 BUSCO variables integrated into BlobTools2

The BUSCO datasets incorporated into BlobTools2 provide information on transcript completeness:

- Complete BUSCOs (C)
- Single-copy BUSCOs (S)
- Duplicated BUSCOs (D)
- Fragmented BUSCOs (F)
- Missing BUSCOs (M)

These metrics facilitate the identification of transcripts that are likely to represent biologically relevant plant genes.

12.3 Integration of Read Coverage Information

Read coverage estimates generated with Minimap2 were incorporated into the BlobDir databases to provide quantitative support for each assembled transcript.

Coverage information is one of the most informative variables in BlobTools2 because contaminants frequently exhibit coverage patterns that differ from those of genuine host transcripts.

Input BAM files were obtained from:

08_minimap2_bams/

The updated BlobDir databases were stored in:

09_BlobTools2_Zamia_RNA/

12.3.1 BlobTools2 coverage integration parameters

Coverage information was added using:

- `--cov`: BAM alignment file generated with Minimap2.
- `--replace`: Replaces any previously loaded coverage dataset.
- `add`: Adds coverage information to an existing BlobDir.

12.3.2 SLURM script used for coverage integration

SLURM script: 10_C_Blob_add_Minimap2.sh

```
#!/bin/bash

#SBATCH -J Minblob_add_cov_Zam
#SBATCH -o Minblob_add_cov_Zamia_RNA_%j.out
#SBATCH -p small
#SBATCH --array=6
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=2
#SBATCH --mem=20G
#SBATCH --time=1-00:00:00
#SBATCH --mail-type=ALL
#SBATCH --mail-user=gfpeb@ulaval.ca

#####
module load miniconda/3
module load blobtools/2
module load python/3.11

#Activate conda environment before running
eval "$(conda shell.bash hook)"
conda activate btk_env

#####
# Rutas base

base_dir="/project/jcvaguilar/users/jcvil9/Gab_Zamia_Transcri"
samples_list="${base_dir}/samples_list.txt"
bam_base="${base_dir}/08_minimap2_bams"
blobdir_base="${base_dir}/09_BlobTools2_Zamia_RNA"

#####
# Loop por muestra

while read -r sample; do
```

```

sample=$(echo "$sample" | tr -d '[:space:]')
echo "Procesando muestra: $sample"

#####
# Buscar BAM
bam_dir=$(find "$bam_base" -type d -name "${sample}_*_minimap2" | sort | tail -n
↵ 1)
bam_file="${bam_dir}/${sample}_assembly.bam"

#####
# Buscar BlobDir
blobdir=$(find "$blobdir_base" -maxdepth 1 -type d -name "${sample}_*_blobdir" |
↵ sort | tail -n 1)

#####
# Aadir cobertura
echo " Aadiendo cobertura para $sample"
/prg/blobtools2/blobtools2/blobtools add \
--replace \
--cov "$bam_file" \
"$blobdir"

echo "Finalizado: $sample"
echo "-----"

done < "$samples_list"

conda deactivate

```

12.3.3 Coverage variables integrated into BlobTools2

The Minimap2 alignments provide:

- Mean read coverage per transcript.
- Total mapped reads.
- Coverage distribution across transcripts.
- Relative abundance of assembled sequences.

These metrics help distinguish highly supported transcripts from low-coverage sequences that may represent assembly artifacts or contaminants.

12.4 Key BlobTools2 Variables for Transcriptome Assessment

The most important variables incorporated into BlobTools2 and used during downstream contamination filtering include:

12.4.1 Taxonomic Assignment

Obtained from BLAST analyses and later integrated into the BlobDir database.

Important fields include:

- Taxonomic identifier (TaxID)
- Scientific name
- Taxonomic lineage
- Assigned kingdom

These variables facilitate the identification of non-target organisms such as fungi, bacteria, viruses, or metazoan contaminants.

12.4.2 GC Content

Represents the percentage of guanine and cytosine nucleotides in each transcript.

Sequences belonging to different taxonomic groups often exhibit distinct GC-content distributions.

12.4.3 Transcript Length

The length of each assembled transcript.

Very short transcripts frequently correspond to fragmented assemblies or low-confidence sequences.

12.4.4 Read Coverage

Derived from Minimap2 alignments.

Coverage values provide evidence supporting transcript validity and expression level.

12.4.5 BUSCO Status

Indicates whether transcripts contain conserved orthologs expected in plants.

BUSCO-supported transcripts are generally more reliable candidates for downstream analyses.

12.5 BlobTools2 Output Files

After all datasets have been incorporated, each BlobDir contains a complete collection of metadata describing the transcriptome assembly.

```
09_BlobTools2_Zamia_RNA/  
  JCV23-0101_20250617_blobdir/  
    meta.json  
    summary.json
```

```
identifiers.json
busco/
coverage/
views/
filters/
...
JCV23-0102_20250617_blobdir/
JCV23-0201_20250617_blobdir/
...
```

These BlobDir databases serve as the primary input for subsequent taxonomic assignment, contamination filtering, visualization, and extraction of Viridiplantae transcripts used in downstream transcriptome analyses.

13 BLASTx Against the *Cycas* Reference Proteome

In addition to the searches against the NCBI NR database, each Trinity assembly was compared against the predicted proteome of *Cycas panzhihuaensis* (*Cycas_panzhihuaensis_genome.pep*). Because *Cycas* and *Zamia* belong to the order Cycadales, this analysis provides an independent assessment of transcriptome completeness and gene recovery using a closely related reference species.

The analysis was performed on the Trinity assemblies located in:

03_trinity_separated/

The reference protein dataset used for comparison was:

Cycas_panzhihuaensis_genome.pep

The output files were stored in:

11_BlastX_GenRefCycas_Zamia_RNA/

13.1 BLASTx command parameters

The BLASTx searches were executed using the following key parameters:

- `-db cycas_pep_db`: Custom BLAST protein database generated from the *Cycas panzhihuaensis* proteome.
- `-max_target_seqs 1`: Retains only the best match for each transcript.
- `-evalue 1e-25`: Stringent significance threshold.
- `-num_threads`: Number of CPU threads allocated by SLURM.
- `-outfmt 6`: Standard tabular BLAST output format.

Unlike the searches against NR, this analysis focused exclusively on identifying the best homologous protein in the reference cycad proteome.

13.2 Protein coverage assessment

Following BLASTx, the script `analyze_blastPlus_topHit_coverage.pl` from the Trinity package was used to evaluate reference protein coverage.

This analysis estimates:

- The proportion of reference proteins represented in the transcriptome assembly.
- The fraction of each protein covered by assembled transcripts.
- The completeness of reconstructed coding sequences.
- The distribution of full-length and partial transcript matches.

These metrics provide an additional measure of transcriptome quality and gene-space recovery.

13.3 SLURM script used for BLASTx against the Cycas reference proteome

SLURM script: `11_Blastx_REFcycas_proteins_Zamia_RNA.sh`

```
#!/bin/bash
#SBATCH -J BlaRefCy_Za
#SBATCH -o blastxRefCy_Zamia-%j.out
#SBATCH -p small
#SBATCH --array=1-12
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --mem=10G
#SBATCH --time=1-00:00:00
#SBATCH --mail-type=ALL
#SBATCH --mail-user=gfpeb@ulaval.ca

#####
module load trinity/2.15.2
module load ncbiblast/2.16.0

##### Rutas generales

base_dir="/project/jcvaguilar/users/jcvil9/Gab_Zamia_Transcri"
samples_list="${base_dir}/samples_list.txt"
trinity_base="${base_dir}/03_trinity_separated"
blastx_base="${base_dir}/11_BlastX_GenRefCycas_Zamia_RNA"
pep_ref="${base_dir}/Cycas_panzhihuaensis_genome.pep/Cycas_panzhihuaensis_genome.pep"

##### Obtener muestra

sample=$(sed -n "${SLURM_ARRAY_TASK_ID}p" "$samples_list" | tr -d '[:space:]')
fecha=$(date +%Y%m%d)
trinity_dir=$(find "$trinity_base" -maxdepth 1 -type d -name "${sample}*_trinity"
↵ | sort | tail -n 1)
```

```

fasta_file=$(find "$trinity_dir" -maxdepth 1 -type f -name "*.Trinity.fasta" |
↪ head -n 1)

##### Crear carpeta de salida

output_dir="${blastx_base}/${sample}_${fecha}_blastx"
mkdir -p "$output_dir"
log_file="$output_dir/${sample}_blastx.log"

##### Crear base de datos BLAST si no existe
# Para proteínas dbtype prot
# Para nucleotidos dbtype nucl

makeblastdb -in "$pep_ref" -dbtype prot -parse_seqids -out
↪ "$output_dir/cycas_pep_db" >> "$log_file" 2>&1

##### Exporta la ruta donde est tu NR, base de datos taxonomicos

export BLASTDB=/biodata/blast

##### Ejecutar BLASTx contra cycas_pep_db

blastx -query "$fasta_file" \
  -db "$output_dir/cycas_pep_db" \
  -max_target_seqs 1 \
  -evalue 1e-25 \
  -num_threads "$SLURM_CPUS_PER_TASK" \
  -outfmt 6 \
  -out "$output_dir/${sample}_blastx_topHit.outfmt6" \
  >> "$log_file" 2>&1

##### Analizar cobertura con Trinity

coverage_script="/prg/trinityrnaseq/2.15.2/util/analyze_blastPlus_topHit_coverage.pl"

"$coverage_script" \
  "$output_dir/${sample}_blastx_topHit.outfmt6" \
  "$pep_ref" \
  "$pep_ref" >> "$log_file" 2>&1

echo "BLASTx y analisis de cobertura completado para $sample"

```

13.4 Output files

```

11_BlastX_GenRefCycas_Zamia_RNA/
  JCV23-0101_20250617_blastx/
    cycas_pep_db.*
    JCV23-0101_blastx_topHit.outfmt6
    JCV23-0101_blastx.log
    blastx_topHit_coverage_report.txt
  ...
  JCV23-0102_20250617_blastx/
  JCV23-0201_20250617_blastx/
  ...

```

The resulting BLAST and coverage statistics were subsequently used to evaluate the representation of conserved cycad genes within the *Zamia* transcriptome assemblies and to identify transcripts corresponding to near full-length protein-coding genes.

14 Coverage-Based and Taxonomic Filtering of Transcript Assemblies

Following transcriptome assembly and similarity searches against the *Cycas panzhihuaensis* reference proteome, a multi-step filtering strategy was applied to retain high-confidence plant transcripts while removing poorly supported or potentially contaminant sequences.

The filtering workflow consisted of three sequential steps:

1. Selection of transcripts with high protein coverage against the *Cycas* reference proteome.
2. Taxonomic assignment of the retained transcripts using BlobTools2 and BLASTx results.
3. Extraction of transcripts assigned to Viridiplantae for downstream analyses.

14.1 Filtering Transcripts by Reference Protein Coverage

The first filtering step retained only transcripts showing substantial alignment coverage against proteins from *Cycas panzhihuaensis*.

Coverage information was obtained from the file generated by Trinity:

```
*.w_pct_hit_length
```

Only transcripts covering at least 80% of the length of their best reference protein hit were retained.

This threshold was chosen to enrich the dataset for nearly complete coding sequences and reduce the number of fragmented transcript assemblies.

14.1.1 Coverage filtering criterion

The filtering was based on:

- Column 14 (pct_hit_len_aligned) from the Trinity coverage report.

- Minimum coverage threshold: 80%.
- Transcript extraction performed using `seqtk subseq`.

14.1.2 SLURM script used for coverage filtering

SLURM script: 12_1_filter_cover_Zamia.sh

```
#!/bin/bash

#SBATCH -J filterCov
#SBATCH -o filterCov_ZamRNA-%j.out
#SBATCH -p small
#SBATCH --array=1-12
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=2
#SBATCH --mem=5G
#SBATCH --time=1-00:00:00
#SBATCH --mail-type=ALL
#SBATCH --mail-user=gfpeb@ulaval.ca

set -euo pipefail

#####

module load trinity/2.15.2
module load ncbiblast/2.16.0
module load seqtk/1.2

#####
# Rutas base

base_dir="/project/jcvaguilar/users/jcvil9/Gab_Zamia_Transcri"
samples_list="${base_dir}/samples_list.txt"
trinity_base="${base_dir}/03_trinity_separated"
blastx_base="${base_dir}/10_BlastX_GenRefCycas_Zamia_RNA"

# Carpeta de salida de los FASTA filtrados por cobertura

filter_root="${base_dir}/11_Filter_topHit_coverage_Zamia_RNA"
mkdir -p "$filter_root"

#####
# Extraer la muestra y fecha

sample=$(sed -n "${SLURM_ARRAY_TASK_ID}p" "$samples_list" | tr -d '[:space:]')
fecha=$(date +%Y%m%d)

#####
```

```

# Localizar archivos

trinity_dir=$(find "$trinity_base" -maxdepth 1 -type d -name "${sample}*_trinity"
↪ | sort | tail -n 1)
fasta_file=$(find "$trinity_dir" -maxdepth 1 -type f -name "*.Trinity.fasta" |
↪ head -n 1)

blastx_dir=$(find "$blastx_base" -maxdepth 1 -type d \
    -name "${sample}*_blastx" | sort | tail -n1)

if [[ -z "$blastx_dir" ]]; then
    echo "ERROR: No encontr ning n directorio *${sample}*_blastx en $blastx_base"
    ↪ >&2
    exit 1
fi

coverage_file="${blastx_dir}/${sample}_blastx_topHit.outfmt6.w_pct_hit_length"

if [[ ! -f "$coverage_file" ]]; then
    echo "ERROR: No se encontr el archivo de cobertura (.w_pct_hit_length):"
    echo "  $coverage_file" >&2
    exit 1
fi

#####
# Preparamos carpeta y log de filtrado

filter_dir="${filter_root}/${sample}_${fecha}_filtered_hitsCover"
mkdir -p "$filter_dir"
log_file="${filter_dir}/${sample}_filterCoverage.log"

# Redirigimos stdout y stderr a log (y pantalla)
exec >>(tee -i "$log_file") 2>&1

echo "[$(date +%F %T)] Filtrando transcritos con cobertura = 80 % para $sample"

#####
# Extraer los IDs con cobertura = 80 % (columna 14 = pct_hit_len_aligned)

awk '$14 >= 80 { print $1 }' "$coverage_file" \
    > "${filter_dir}/${sample}_80cov.ids"

n_ids=$(wc -l < "${filter_dir}/${sample}_80cov.ids")
echo "Encontrados $n_ids transcritos con = 80 % de cobertura."

#####
# Sub-secuenciar el FASTA original

```

```
seqtk subseq "$fasta_file" "${filter_dir}/${sample}_80cov.ids" \
> "${filter_dir}/${sample}_80cov.fasta"

echo "[$(date +%F %T)] Filtrado completado."
echo "Resultado: ${filter_dir}/${sample}_80cov.fasta"
```

14.1.3 Output files

```
11_Filter_topHit_coverage_Zamia_RNA/
  JCV23-0101_20250617_filtered_hitsCover/
    JCV23-0101_80cov.ids
    JCV23-0101_80cov.fasta
    JCV23-0101_filterCoverage.log
  JCV23-0102_20250617_filtered_hitsCover/
  JCV23-0201_20250617_filtered_hitsCover/
  ...
```

The resulting FASTA files contain transcripts supported by strong protein-level evidence from a closely related cycad species.

14.2 Taxonomic Assignment of High-Coverage Transcripts

The filtered transcript assemblies were subsequently analyzed using BlobTools2.

A new BlobDir database was generated from each filtered FASTA file and populated with taxonomic information derived from the original BLASTx searches against the NR database.

This step allowed the assignment of each retained transcript to a taxonomic lineage.

Input files included:

- Filtered FASTA files from Step 1.
- BLASTx results against NR115.
- NCBI taxonomy database (`taxdump`).

14.2.1 BlobTools2 taxonomic assignment parameters

The following parameters were used:

- `--hits`: BLASTx results against NR115.
- `--taxdump`: NCBI taxonomy database.
- `--taxrule bestsumorder`: BlobTools2 taxonomic assignment algorithm.
- `--evaluate 1e-25`: Minimum significance threshold.
- `--replace`: Replaces previous taxonomic assignments if present.

14.2.1.1 Taxonomic assignment method BlobTools2 uses the **bestsumorder** algorithm, which assigns taxonomy by evaluating all significant BLAST hits associated with a transcript and selecting the most supported taxonomic lineage.

This approach is generally more robust than relying solely on the top BLAST hit and reduces misclassification caused by isolated matches.

14.2.2 SLURM script used for taxonomic assignment

SLURM script: 12_2_Blob_CreAdd_filter_Cover_Zamia.sh

```
#!/bin/bash

#SBATCH -J Blob-filter
#SBATCH -o BlobFilter_ZamRNA-%j.out
#SBATCH -p small
#SBATCH --array=1-12
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=2
#SBATCH --mem=15G
#SBATCH --time=1-00:00:00
#SBATCH --mail-type=ALL
#SBATCH --mail-user=gfpeb@ulaval.ca

#####
module load miniconda/3
module load blobtools/2
module load python/3.11
module load seqtk/1.2

#Activate conda environment before running
eval "$(conda shell.bash hook)"
conda activate btk_env

#### Rutas base

base_dir="/project/jcvaguilar/users/jcvil9/Gab_Zamia_Transcri"
samples_list="${base_dir}/samples_list.txt"
filter_root="$base_dir/11_Filter_topHit_coverage_Zamia_RNA"
blastx_base="$base_dir/06_BlastX_Zamia_RNA"
taxdump_dir="$base_dir/new_taxdump"
blobtools_root="$base_dir/12_BlobTools2_FilterCoverage_Zamia_RNA"

mkdir -p "$blobtools_root"

#### Muestra y fecha
```

```

sample=$(sed -n "${SLURM_ARRAY_TASK_ID}p" "$samples_list" | tr -d '[:space:]')
fecha=$(date +%Y%m%d)

#### Localizar FASTA filtrado (cobertura = 80 %)

filter_dir="$filter_root/${sample}_${fecha}_filtered_hitsCover"
filtered_fasta="$filter_dir/${sample}_80cov.fasta"

if [[ ! -f "$filtered_fasta" ]]; then
    echo "ERROR: FASTA filtrado no encontrado:" >&2
    echo "  $filtered_fasta" >&2
    exit 1
fi

#####
# Preparar BlobDir y log en la salida de la muestra

blobdir_out="$blobtools_root/${sample}_${fecha}_filteredBlobDir"
mkdir -p "$blobdir_out"

log_file="$blobdir_out/${sample}_blobtools.log"
exec >>(tee -i "$log_file") 2>&1
echo "[$(date +%F %T)] Iniciando BlobTools2 para $sample"

#####
# Crear BlobDir a partir del FASTA filtrado

/prg/blobtools/2/blobtools2/blobtools create \
  --fasta "$filtered_fasta" \
  "$blobdir_out"

#####
# Subconjuntar hits NR115 originales

nr_dir=$(find "$blastx_base" -maxdepth 1 -type d -name "${sample}_*_blastx_nr115"
↪ | tail -n1)
hit_file="$nr_dir/${sample}_blastx.outfmt6"

if [[ ! -f "$hit_file" ]]; then
    echo "ERROR: Hits NR115 no encontrados:" >&2
    echo "  $hit_file" >&2
    exit 1
fi

#### 2.1) Subconjuntar los hits de NR115 a los contigs de =80 %
ids_file="$filter_dir/${sample}_80cov.ids"
filtered_hits="$blobdir_out/${sample}_blastx_NR115_filtered.outfmt6"

```



```

grep -F -w -f "$ids_file" "$hit_file" > "$filtered_hits"
echo "    Hits de NR115 reducidos a $(wc -l < "$filtered_hits") contigs filtrados"

#####
# Aadir clasificaci n taxon mica de NR115 al BlobDir

/prg/blobtools/2/blobtools2/blobtools add \
  --hits "$filtered_hits" \
  --hits-cols
  ↪ 1=qseqid,2=staxids,3=bitscore,4=sseqid,5=sscinames,6=pident,7=length,8=mismatch,9=gapopen
  ↪ \
  --taxdump "$taxdump_dir" \
  --taxrule bestsumorder \
  --evaluate 1e-25 \
  --replace \
  "$blobdir_out"

echo "[$(date +%F %T)] BlobTools2 completado para $sample"
conda deactivate

```

14.2.3 Output files

```

12_BlobTools2_FilterCoverage_Zamia_RNA/
  JCV23-0101_20250617_filteredBlobDir/
    meta.json
    identifiers.json
    summary.json
    JCV23-0101_blastx_NR115_filtered.outfmt6
    ...
  JCV23-0102_20250617_filteredBlobDir/
  JCV23-0201_20250617_filteredBlobDir/
  ...

```

14.3 Extraction of Viridiplantae Transcripts

The final filtering step retained only transcripts assigned to Viridiplantae.

Using BlobTools2, all transcripts classified outside the plant kingdom were excluded. The resulting identifiers were then used to extract the corresponding sequences from the filtered FASTA files.

This procedure generated the final set of transcripts used in downstream analyses.

14.3.1 BlobTools2 filtering parameters

The filtering was based on:

- Taxonomic rank: Kingdom.
- Target group: Viridiplantae.
- Taxonomic assignments generated using the bestsumorder algorithm.

14.3.1.1 Why filter by Viridiplantae? Although transcriptomes are derived from plant tissues, RNA extractions frequently contain transcripts originating from:

- Endophytic fungi.
- Epiphytic microorganisms.
- Bacteria.
- Viruses.
- Environmental contaminants.

Retaining only sequences assigned to Viridiplantae increases confidence that downstream analyses focus on host plant transcripts.

14.3.2 SLURM script used for Viridiplantae filtering

SLURM script: 12_3_Blob_filter_Streptophyta_Zamia.sh

```
#!/bin/bash

#SBATCH -J Blob-filter
#SBATCH -o BlobFilter_ZamRNA-%j.out
#SBATCH -p small
#SBATCH --array=1-12
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=2
#SBATCH --mem=15G
#SBATCH --time=1-00:00:00
#SBATCH --mail-type=ALL
#SBATCH --mail-user=gfpeb@ulaval.ca

#####
# 1. Activar entorno y cargar módulos
#####

module load miniconda/3
module load blobtools/2
module load python/3.11
module load seqtk/1.2
```

```

#Activate conda environment before running
eval "$(conda shell.bash hook)"
conda activate btk_env

#####
# 2. Rutas base
#####

base_dir="/project/jcvaguilar/users/jcvil9/Gab_Zamia_Transcri"
samples_list="${base_dir}/samples_list.txt"
blobtools_root="${base_dir}/12_BlobTools2_FilterCoverage_Zamia_RNA"
filter_root="${base_dir}/11_Filter_topHit_coverage_Zamia_RNA"
filtered_output_root="${base_dir}/13_BlobTools2_FilterKingdom_Zamia_RNA"

mkdir -p "$filtered_output_root"

#####
# 3. Muestra y fecha
#####

sample=$(sed -n "${SLURM_ARRAY_TASK_ID}p" "$samples_list" | tr -d '[:space:]')
fecha=$(date +%Y%m%d)

#####
# 4. Rutas específicas
#####
# Localizar el BlobDir (ignorar fecha)
blobdir_in=$(find "$blobtools_root" -maxdepth 1 -type d -name
↳ "${sample}_*_filteredBlobDir" | sort | tail -n 1)

# Localizar el directorio del FASTA filtrado por cobertura (ignorar fecha)
filter_dir=$(find "$filter_root" -maxdepth 1 -type d -name
↳ "${sample}_*_filtered_hitsCover" | sort | tail -n 1)
filtered_fasta=$(find "$filter_dir" -type f -name "${sample}_80cov.fasta" | head
↳ -n 1)

# Definir carpeta de salida
filtered_out="${filtered_output_root}/${sample}_${fecha}_filtered_Viridiplantae"

log_file="${filtered_out}/${sample}_blobtools_filter.log"
mkdir -p "$filtered_out"

#####
# 5. Iniciar log
#####

exec >>(tee -i "$log_file") 2>&1
echo "[$(date +%F %T)] Iniciando filtro para conservar solo Viridiplantae en
↳ $sample"

```

```

echo "BlobDir In: $blobdir_in"
echo "Filtered FASTA: $filtered_fasta"
echo "Filtered Out Dir: $filtered_out"

#####
# 6. Ejecutar filtro por Viridiplantae
#####

/prg/blobtools/2/blobtools2/blobtools filter \
  --param bestsumorder_kingdom--Keys=Viridiplantae \
  --invert \
  --fasta "$filtered_fasta" \
  --summary "$filtered_out/summary.json" \
  --table "$filtered_out/table.tsv" \
  --output "$filtered_out" \
  "$blobdir_in"

#####
# 7. Extraer lista de IDs desde identifiers.json
#####

id_json="$filtered_out/identifiers.json"
id_txt="$filtered_out/${sample}_Viridiplantae_ids.txt"
fasta_out="$filtered_out/${sample}_Viridiplantae_filtered.fasta"

if [[ ! -f "$id_json" ]]; then
  echo "[ERROR] No se encontró el archivo identifiers.json" >&2
  exit 1
fi

python3 -c "
import json
with open('$id_json') as f:
    data = json.load(f)
with open('$id_txt', 'w') as out:
    for identifier in data.get('values', []):
        out.write(identifier + '\n')
"

if [[ ! -s "$id_txt" ]]; then
  echo "[ERROR] El archivo de IDs está vacío o falló la conversión." >&2
  exit 1
fi

echo " • $(wc -l < "$id_txt") identificadores extraídos a $id_txt"

#####
# 8. Extraer secuencias FASTA

```

```
#####

seqtk subseq "$filtered_fasta" "$id_txt" > "$fasta_out"

if [[ $? -ne 0 ]]; then
    echo "[ERROR] Falló la extracción de secuencias con seqtk." >&2
    exit 1
fi

echo "    • FASTA filtrado generado en $fasta_out"
echo "[$(date +%F %T)] Proceso completado para $sample"

conda deactivate
```

14.3.3 Output files

```
13_BlobTools2_FilterKingdom_Zamia_RNA/
  JCV23-0101_20250617_filtered_Viridiplantae/
    identifiers.json
    summary.json
    table.tsv
    JCV23-0101_Viridiplantae_ids.txt
    JCV23-0101_Viridiplantae_filtered.fasta
    JCV23-0101_blobtools_filter.log
  JCV23-0102_20250617_filtered_Viridiplantae/
  JCV23-0201_20250617_filtered_Viridiplantae/
  ...
```

14.3.4 Final Output

The final FASTA files generated in this step represent a curated transcriptome dataset composed of:

- Transcripts with strong protein-level support (80% coverage).
- Sequences with significant similarity to known proteins.
- Transcripts taxonomically assigned to Viridiplantae.

These filtered assemblies were used as the starting point for downstream functional annotation and transcriptome analyses.

15 TransRate Assessment of Filtered Transcriptomes

After coverage-based and taxonomic filtering, the final Viridiplantae transcriptomes were evaluated using TransRate. This analysis assesses transcriptome quality by mapping the original RNA-seq

bestsumorder_class.json	bestsumorder_order_score.json	JCV23-0101_assembly_cov.json
bestsumorder_class_cindex.json	bestsumorder_phylum.json	JCV23-0101_assembly_read_cov.json
bestsumorder_class_positions.json	bestsumorder_phylum_cindex.json	JCV23-0101_blobtools_20250626.log
bestsumorder_class_score.json	bestsumorder_phylum_positions.json	length.json
bestsumorder_family.json	bestsumorder_phylum_score.json	meta.json
bestsumorder_family_cindex.json	bestsumorder_positions.json	ncount.json
bestsumorder_family_positions.json	bestsumorder_species.json	viridiplantae_odb10_busco.json
bestsumorder_family_score.json	bestsumorder_species_cindex.json	
bestsumorder_genus.json	bestsumorder_species_positions.json	
bestsumorder_genus_cindex.json	bestsumorder_species_score.json	
bestsumorder_genus_positions.json	bestsumorder_superkingdom.json	
bestsumorder_genus_score.json	bestsumorder_superkingdom_cindex.json	
bestsumorder_kingdom.json	bestsumorder_superkingdom_positions.json	
bestsumorder_kingdom_cindex.json	bestsumorder_superkingdom_score.json	
bestsumorder_kingdom_positions.json	embryophyta_odb10_busco.json	
bestsumorder_kingdom_score.json	gc.json	
bestsumorder_order.json	identifiers.json	
bestsumorder_order_cindex.json	JCV23-0101_add_busco_cleaned.log	
bestsumorder_order_positions.json	JCV23-0101_add_taxonomy.log	

Figure 3: list of output files BlobTools2 Filter.

reads back to the assembled transcripts and calculating several assembly support metrics.

Unlike BUSCO, which evaluates completeness using conserved orthologs, TransRate evaluates how well the sequencing reads support the assembled transcripts.

The analysis was performed using:

- Filtered Viridiplantae transcriptomes generated by BlobTools2.
- Original cleaned paired-end reads generated after Trimmomatic filtering.

Input files were obtained from:

13_BlobTools2_FilterKingdom_Zamia_RNA/

The resulting TransRate reports were stored in:

14_Transrate_FilteredFASTA/

15.1 TransRate quality metrics

TransRate calculates several assembly-support statistics, including:

- Read mapping rate.
- Contig coverage.
- Contig score.
- Assembly score.
- Number of supported transcripts.
- Number of poorly supported transcripts.

These metrics help identify assembly artifacts and provide an independent evaluation of transcriptome quality.

15.1.1 Interpretation of TransRate scores

Important metrics to consider include:

Metric	Interpretation
Assembly score	Overall quality of the transcriptome assembly
Mapping rate	Percentage of reads that align back to assembled transcripts
Good contigs	Number of transcripts strongly supported by reads
Low-supported contigs	Potential assembly artifacts or low-confidence transcripts

Higher assembly scores and mapping rates generally indicate more reliable transcriptome assemblies.

15.2 SLURM script used for TransRate analysis

SLURM script: 13_TransRate_FilterZamia_RNA.sh

```
#!/bin/bash

#SBATCH -J TraRate_Zam
#SBATCH -o TransRate_Zam_test.out
#SBATCH -p small
#SBATCH --cpus-per-task=2
#SBATCH --array=1-5,7-12
#SBATCH --mem=10G
#SBATCH --time=1-00:00:00
#SBATCH --mail-type=ALL
#SBATCH --mail-user=gfpeb@ulaval.ca

#####
# 1. Cargar módulos necesarios
#####

module load usearch/9.2.64_i86linux32
module load bowtie/2.3.4.1
module load eXpress/1.5.1
module load salmon/0.11.0
module load transrate/1.0.3

#####
# 2. Definir rutas base
#####
```

```

base_dir="/project/jcvaguilar/users/jcvil9/Gab_Zamia_Transcri"
samples_list="${base_dir}/samples_list.txt"
reads_base="${base_dir}/01_clean_reads_Zamia_RNA_trimmomatic"
filtered_fasta_base="${base_dir}/13_BlobTools2_FilterKingdom_Zamia_RNA"
results_base="${base_dir}/14_Transrate_FilteredFASTA"

mkdir -p "$results_base"

#####
# 3. Obtener nombre de muestra
#####

sample=$(sed -n "${SLURM_ARRAY_TASK_ID}p" "$samples_list" | tr -d '[:space:]')
fecha=$(date +%Y%m%d)

#####
# 4. Localizar archivo FASTA filtrado por reino
#####

filtered_fasta_dir=$(find "$filtered_fasta_base" -maxdepth 1 -type d -name
↪ "${sample}_*_filtered_Viridiplantae*" | sort | tail -n 1)
filtered_fasta=$(find "$filtered_fasta_dir" -maxdepth 1 -type f -name
↪ "${sample}_Viridiplantae_filtered.fasta" | head -n 1)

if [[ ! -f "$filtered_fasta" ]]; then
    echo "[ERROR] No se encontró el archivo FASTA filtrado por reino para $sample"
    ↪ >&2
    exit 1
fi

#####
# 5. Definir archivos de lectura y salida
#####

left_reads="${reads_base}/${sample}/${sample}_R1_paired.fastq.gz"
right_reads="${reads_base}/${sample}/${sample}_R2_paired.fastq.gz"
output_dir="${results_base}/${sample}_${fecha}_transrate_filtered"
log_file="${output_dir}/${sample}_transrate_filtered.log"

mkdir -p "$output_dir"

#####
# 6. Ejecutar Transrate con archivo filtrado
#####

transrate -a "$filtered_fasta" \
    --left "$left_reads" \
    --right "$right_reads" \

```



```

-t 2 \
--output "$output_dir" &> "$log_file"

echo "[INFO] Transrate completado para $sample"

```

15.3 Output files

```

14_Transrate_FilteredFASTA/
  JCV23-0101_20250617_transrate_filtered/
    assemblies.csv
    contigs.csv
    scores.csv
    transrate.log
    ...
  JCV23-0102_20250617_transrate_filtered/
  JCV23-0201_20250617_transrate_filtered/
  ...

```

The TransRate reports provide a final quality assessment of the filtered transcriptomes before coding-sequence prediction and functional annotation.

16 TransDecoder: Prediction of Coding Sequences

Following transcriptome quality assessment, TransDecoder was used to identify candidate coding regions (Open Reading Frames; ORFs) within the filtered Viridiplantae transcriptomes.

TransDecoder predicts likely protein-coding sequences by identifying long open reading frames and evaluating coding potential.

This step generates the protein datasets required for downstream functional annotation and orthology analyses.

Input files were obtained from:

```
13_BlobTools2_FilterKingdom_Zamia_RNA/
```

The predicted coding sequences were stored in:

```
15_TransDecoder_Zamia_RNA/
```

16.1 TransDecoder workflow

The analysis was performed in two stages:

16.1.1 TransDecoder.LongOrfs

Identifies candidate open reading frames within transcript sequences.

16.1.2 TransDecoder.Predict

Evaluates candidate ORFs and predicts the most likely coding regions.

16.2 Key output files

TransDecoder generates several useful files:

File	Description
.pep	Predicted protein sequences
.cds	Predicted coding sequences
.gff3	Coordinates of predicted ORFs
.bed	Genomic coordinates of ORFs

Among these, the protein FASTA (.pep) file is typically used for downstream annotation workflows.

16.3 SLURM script used for TransDecoder

SLURM script: 14_TransDecoder_Zamia_RNA.sh

```
#!/bin/bash

#SBATCH -J Tdecoder-zamia
#SBATCH -o TransDecoder_Zam_test.out
#SBATCH -p small
#SBATCH --cpus-per-task=2
#SBATCH --array=1-12
#SBATCH --mem=10G
#SBATCH --time=1-00:00:00
#SBATCH --mail-type=ALL
#SBATCH --mail-user=gfpeb@ulaval.ca

#####

module load transdecoder/5.5.0
module load usearch/9.2.64_i86linux32

#####
# 1. Rutas base
#####

base_dir="/project/jcvaguilar/users/jcvil9/Gab_Zamia_Transcri"
samples_list="${base_dir}/samples_list.txt"

# Entrada: FASTA filtrado por BlobTools2
```

```

viridi_root="${base_dir}/13_BlobTools2_FilterKingdom_Zamia_RNA"

# Salida: carpeta final organizada
transdecoder_root="${base_dir}/15_TransDecoder_Zamia_RNA"
tabla_out="${transdecoder_root}/resumen_transdecoder.tsv"

mkdir -p "$transdecoder_root"

if [[ "$SLURM_ARRAY_TASK_ID" == 1 && ! -f "$tabla_out" ]]; then
    echo -e "sample\ttotal_orfs\tmin_len\tmax_len\tavg_len" > "$tabla_out"
fi

#####
# 2. Identificar muestra
#####

sample=$(sed -n "${SLURM_ARRAY_TASK_ID}p" "$samples_list" | tr -d '[:space:]')
fecha=$(date +%Y%m%d)

viridi_dir=$(find "$viridi_root" -maxdepth 1 -type d -name
    ↪ "${sample}_*_filtered_Viridiplantae" | sort | tail -n 1)
fasta_in="${viridi_dir}/${sample}_Viridiplantae_filtered.fasta"

if [[ ! -f "$fasta_in" ]]; then
    echo "[ERROR] FASTA no encontrado: $fasta_in"
    exit 1
fi

#####
# 3. Crear carpeta de trabajo y moverse all
#####

td_out="${transdecoder_root}/${sample}_${fecha}_transdecoder_out"
mkdir -p "$td_out"
cd "$td_out" || exit 1

#####
# 4. Ejecutar TransDecoder desde output_dir
#####

echo "[$(date +%F %T)] Procesando $sample"

TransDecoder.LongOrfs -t "$fasta_in"
TransDecoder.Predict -t "$fasta_in"

#####
# 5. Extraer m tricas del archivo .pep
#####

```

```

pep_file="${td_out}/${basename "$fasta_in").transdecoder.pep"

if [[ ! -f "$pep_file" ]]; then
    echo "[ERROR] No se encontr el archivo .pep para $sample"
    exit 1
fi

lens=$(awk '/^>/ {if (seq) print length(seq); seq=""} !/^>/ {seq=seq $0}
↪ END{print length(seq)}' "$pep_file")

total=${#lens[@]}
min=${lens[0]}
max=${lens[0]}
sum=0

for len in "${lens[@]"; do
    (( sum += len ))
    (( len < min )) && min=$len
    (( len > max )) && max=$len
done

avg=$(awk -v s=$sum -v t=$total 'BEGIN{printf "%.2f", s/t}')

# Escribir en la tabla resumen
{
    flock 200
    echo -e "${sample}\t${total}\t${min}\t${max}\t${avg}" >> "$tabla_out"
} 200>"${tabla_out}.lock"

echo " TransDecoder finalizado para $sample"

```

16.4 Summary statistics generated

For each sample, a summary table was generated containing:

- Total number of predicted ORFs.
- Minimum protein length.
- Maximum protein length.
- Mean protein length.

These metrics provide a general overview of the coding potential recovered from each transcriptome assembly.

16.5 Output files

```
15_TransDecoder_Zamia_RNA/  
  resumen_transdecoder.tsv  
  JCV23-0101_20250617_transdecoder_out/  
    JCV23-0101_Viridiplantae_filtered.fasta.transdecoder.pep  
    JCV23-0101_Viridiplantae_filtered.fasta.transdecoder.cds  
    JCV23-0101_Viridiplantae_filtered.fasta.transdecoder.gff3  
    ...  
  JCV23-0102_20250617_transdecoder_out/  
  JCV23-0201_20250617_transdecoder_out/  
  ...
```

The predicted protein sequences generated by TransDecoder constitute the final coding-sequence dataset and serve as the primary input for functional annotation, orthology inference, and comparative transcriptomic analyses.

17 TransDecoder Analysis of the Global Trinity Assembly

In addition to the per-sample analyses, TransDecoder was also applied to the global transcriptome assembly generated from all RNA-seq samples combined. This approach allows the identification of coding regions from the complete transcriptome dataset and provides a comprehensive protein-coding reference for downstream analyses.

The input transcriptome corresponded to the global Trinity assembly generated from all cleaned reads:

```
03_trinity_separated/Zamia_all_20250902_trinity/
```

The predicted coding sequences were stored in:

```
18_B_TransDecoder_All_Zamia_Trinity/
```

17.1 Purpose of the global analysis

Running TransDecoder on the combined assembly offers several advantages:

- Identification of coding regions from the complete transcriptome.
- Recovery of transcripts that may be absent or fragmented in individual assemblies.
- Generation of a comprehensive protein dataset for functional annotation.
- Creation of a reference protein catalog for comparative and evolutionary analyses.

17.2 TransDecoder workflow

The analysis followed the standard two-step TransDecoder workflow.

17.2.1 TransDecoder.LongOrfs

Identifies candidate open reading frames (ORFs) from the assembled transcripts.

17.2.2 TransDecoder.Predict

Evaluates the candidate ORFs and predicts the most likely protein-coding sequences.

17.3 Summary statistics generated

After ORF prediction, a summary table was automatically generated containing:

- Total number of predicted ORFs.
- Minimum protein length.
- Maximum protein length.
- Average protein length.

These metrics provide a general overview of the coding potential recovered from the global transcriptome assembly.

17.4 SLURM script used for global TransDecoder analysis

SLURM script: 17_B_TransDecoder_ALLZamia_trinity_RNA.sh

```
#!/bin/bash

#SBATCH -J Tdeco-Allzamia
#SBATCH -o TransDecoder_AllZam.out
#SBATCH -p medium
#SBATCH --cpus-per-task=5
#SBATCH --mem=30G
#SBATCH --time=7-00:00:00
#SBATCH --mail-type=ALL
#SBATCH --mail-user=gfpeb@ulaval.ca

#####

module load transdecoder/5.5.0
module load usearch/9.2.64_i86linux32

#####
# 1. Rutas
#####

base_dir="/project/jcvaguilar/users/jcvil9/Gab_Zamia_Transcri"
```

```

fasta_global="${base_dir}/03_trinity_separated/Zamia_all_20250902_trinity/Zamia_all_20250902.Tr
out_dir="${base_dir}/18_B_TransDecoder_All_Zamia_Trinity"

mkdir -p "$out_dir"
cd "$out_dir" || exit 1

#####
# 2. Ejecutar TransDecoder
#####

echo "[$(date +%F %T)] Iniciando TransDecoder para FASTA global"

# Paso 1: Long ORFs
TransDecoder.LongOrfs -t "$fasta_global"

# Paso 2: Predict
TransDecoder.Predict -t "$fasta_global"

echo "[$(date +%F %T)] TransDecoder finalizado."

#####
# 3. Estad sticas del .pep
#####

pep_file="$(basename "$fasta_global").transdecoder.pep"
resumen_out="${out_dir}/resumen_transdecoder.tsv"

if [[ ! -f "$pep_file" ]]; then
    echo "[ERROR] No se encontr el archivo .pep"
    exit 1
fi

lens=$(awk '/^>/ {if (seq) print length(seq); seq=""} !/^>/ {seq=seq $0}
↪ END{print length(seq)}' "$pep_file"))

total=${#lens[@]}
min=${lens[0]}
max=${lens[0]}
sum=0

for len in "${lens[@]}"; do
    (( sum += len ))
    (( len < min )) && min=$len
    (( len > max )) && max=$len
done

avg=$(awk -v s=$sum -v t=$total 'BEGIN{printf "%.2f", s/t}')

```

```
echo -e "total_orfs\tmin_len\tmax_len\tavg_len" > "$resumen_out"
echo -e "${total}\t${min}\t${max}\t${avg}" >> "$resumen_out"

echo "? Tabla resumen generada: $resumen_out"
```

17.5 Output files

```
18_B_TransDecoder_All_Zamia_Trinity/
  Zamia_all_20250902.Trinity.fasta.transdecoder.pep
  Zamia_all_20250902.Trinity.fasta.transdecoder.cds
  Zamia_all_20250902.Trinity.fasta.transdecoder.gff3
  Zamia_all_20250902.Trinity.fasta.transdecoder.bed
  longest_orfs.pep
  longest_orfs.cds
  resumen_transdecoder.tsv
  ...
```

17.5.1 Description of the main output files

File	Description
.pep	Predicted protein sequences translated from candidate ORFs
.cds	Coding nucleotide sequences corresponding to predicted proteins
.gff3	Coordinates of predicted ORFs within transcripts
.bed	BED-format coordinates of coding regions
resumen_transdecoder.tsv	Summary statistics of predicted proteins

The resulting protein dataset represents the predicted coding component of the complete *Zamia* transcriptome and can be used as input for functional annotation pipelines such as Trinotate, EggNOG, InterProScan, or comparative genomics analyses.

18 Trinotate: Functional Annotation of the Global Transcriptome

Trinotate is a comprehensive annotation framework designed to provide functional characterization of transcriptome assemblies. It integrates sequence similarity searches, protein domain identification, signal peptide prediction, Gene Ontology (GO) terms, and functional databases into a single SQLite database and annotation report.

Unlike the previous analyses, which were executed on the HPC cluster, the Trinotate workflow was performed locally on a workstation using the official Docker image provided by the Trinotate

developers. This approach simplifies software installation and ensures compatibility with all required dependencies.

For additional information, see:

- Trinotate Wiki: <https://github.com/Trinotate/Trinotate/wiki>
- Trinotate via Docker: <https://github.com/Trinotate/Trinotate/wiki/TrinotateViaDocker>

18.1 Input files

The annotation was performed using the global Trinity assembly and the predicted coding sequences generated by TransDecoder.

Input files

```
Zamia_all_20250902.Trinity.fasta
Zamia_all_20250902.Trinity.fasta.gene_trans_map
Zamia_all_20250902.Trinity.fasta.transdecoder.pep
uniprot_sprot.ncbi.blastx.outfmt6
uniprot_sprot.ncbi.blastp.outfmt6
TrinotatePFAM_B_dombtblout
signalp.out
```

18.2 Trinotate databases and annotation sources

Trinotate integrates several complementary sources of biological information:

Resource	Description
SwissProt BLASTX	Similarity between transcripts and curated protein sequences.
SwissProt BLASTP	Similarity between predicted proteins and curated protein sequences.
Pfam	Conserved protein domains identified with HMMER.
SignalP	Prediction of secreted proteins and signal peptides.
Gene Ontology (GO)	Functional annotation of biological processes, molecular functions, and cellular components.
EggNOG	Orthology and functional classification database.

18.3 Docker environment

The official Docker image was used to execute Trinotate and maintain a reproducible environment.

```
docker run -it --rm \
-v "Trinotate-v4.0.2:/usr/local/src/Trinotate" \
-v "databases:/databases" \
-v "ZamiaAll_trinity:/data" \
trinotatrnaseq/trinotate:latest /bin/bash
```

18.4 Creation of the Trinotate database

A local SQLite database was generated using the most recent Trinotate release (v4.0.2).

```
/usr/local/src/Trinotate/Trinotate \  
--db /data/Trinotate_03Mar2026.sqlite \  
--create \  
--trinotate_data_dir /databases
```

18.4.1 EggNOG database issue

During database construction, the automatic EggNOG download failed because the URL used by Trinotate was no longer active. To resolve this issue:

1. The EggNOG annotation file was downloaded manually.
2. The corresponding checkpoint file was generated.
3. The download command inside `Build_Trinotate_Boilerplate_SQLite_db.pl` was temporarily disabled. Line `# pipeline - > add_commands(newCommand("wget EGGNOG_DAT_URL" -O NOG.annotations.tsv.gz", "eggnog_download.ok"))`);
4. The database creation process was resumed.

This modification allowed successful completion of the SQLite database construction.

18.5 Initialization of the transcriptome database

The Trinity assembly, transcript-to-gene map, and predicted proteins were loaded into the newly created Trinotate database.

```
/usr/local/src/Trinotate/Trinotate \  
--db Trinotate_03Mar2026.sqlite \  
--init \  
--gene_trans_map Zamia_all_20250902.Trinity.fasta.gene_trans_map \  
--transcript_fasta Zamia_all_20250902.Trinity.fasta \  
--transdecoder_pep Zamia_all_20250902.Trinity.fasta.transdecoder.pep
```

18.6 Loading functional annotation results

The different annotation datasets were incorporated into the SQLite database.

18.6.1 BLASTX against SwissProt

```
/usr/local/src/Trinotate/Trinotate \  
--db Trinotate_03Mar2026.sqlite \  
--LOAD_swissprot_blastx \  
uniprot_sprot.ncbi.blastx.outfmt6
```

Number of BLASTX annotations loaded:

235,669 hits

18.6.2 BLASTP against SwissProt

```
/usr/local/src/Trinotate/Trinotate \  
  --db Trinotate_03Mar2026.sqlite \  
  --LOAD_swissprot_blastp \  
  uniprot_sprot.ncbi.blastp.outfmt6
```

Number of BLASTP annotations loaded:

321,403 hits

18.6.3 Pfam domains

```
/usr/local/src/Trinotate/Trinotate \  
  --db Trinotate_03Mar2026.sqlite \  
  --LOAD_pfam \  
  TrinotatePFAM_B_domtblout
```

Number of Pfam domain annotations:

386 hits

18.6.4 SignalP predictions

```
/usr/local/src/Trinotate/Trinotate \  
  --db Trinotate_03Mar2026.sqlite \  
  --LOAD_signalp \  
  signalp.out
```

Number of SignalP annotations:

4,166 hits

18.6.5 EggNOG annotations

```

/usr/local/src/Trinotate/Trinotate \
--db Trinotate_03Mar2026.sqlite \
--LOAD_EggnogMapper \
NOG.annotations.tsv.gz

```

Although the EggNOG table was successfully loaded, only a limited number of entries were recovered. Additional annotation using the current version of EggNOG-mapper may improve functional coverage.

18.7 Gene Ontology verification

The Gene Ontology tables were successfully populated during database creation.

```

sqlite3 Trinotate_03Mar2026.sqlite \
"SELECT COUNT(*) FROM go;"

sqlite3 Trinotate_03Mar2026.sqlite \
"SELECT COUNT(*) FROM go_slim_mapping;"

```

Results:

GO terms:	48,251
GO slim mappings:	35,338

These values indicate that GO annotations were correctly incorporated into the Trinotate database and are available for downstream functional analyses.

18.8 Generation of the final annotation report

The final annotation report was generated directly from the SQLite database.

```

/usr/local/src/Trinotate/Trinotate \
--db /data/Trinotate_03Mar2026.sqlite \
--report \
--incl_pep \
--incl_trans \
> /data/Trinotate_FINAL_04Mar2026.xls

```

18.9 Output files

```
16_Trinotate_Zamia_RNA/  
  Trinotate_03Mar2026.sqlite  
  Trinotate_FINAL_04Mar2026.xls  
  uniprot_sprot.ncbi.blastx.outfmt6  
  uniprot_sprot.ncbi.blastp.outfmt6  
  TrinotatePFAM_B_domtblout  
  signalp.out  
  annotation_logs/
```

The final Trinotate report integrates sequence similarity, protein domains, signal peptide predictions, orthology information, and Gene Ontology assignments, providing a comprehensive functional annotation resource for the global *Zamia* transcriptome.

19 Salmon: Transcript Abundance Estimation

Salmon is a fast and accurate tool for transcript quantification from RNA-seq data. It estimates transcript abundance by quasi-mapping sequencing reads to a reference transcriptome and reports normalized expression values such as TPM (*Transcripts Per Million*).

In this analysis, transcript abundance was estimated by mapping each sample against the global Trinity assembly generated from all *Zamia* samples. This approach allows expression values to be directly compared across samples because all reads are quantified against the same transcript reference.

For more information, see:

- Salmon: <https://combine-lab.github.io/salmon/>

19.1 Input files

The analysis used the global Trinity assembly as the reference transcriptome and the cleaned paired-end reads generated after Trimmomatic filtering.

```
Input files  
  Zamia_all_20250902.Trinity.fasta  
  Zamia_all_20250902.Trinity.fasta.gene_trans_map  
  sample_R1_paired.fastq.gz  
  sample_R2_paired.fastq.gz
```

19.2 Output directory

All abundance estimates and expression matrices were stored in:

19.3 Salmon workflow

The workflow consisted of three main steps:

1. Construction of a Salmon index from the global Trinity assembly.
2. Quantification of transcript abundance for each sample independently.
3. Generation of transcript-level and gene-level abundance matrices across all samples.

19.4 Important parameters

The following parameters were used during quantification:

Parameter	Description
<code>salmon index</code>	Builds an index of the transcriptome assembly.
<code>-k 31</code>	K-mer size used for indexing.
<code>-l A</code>	Automatically detects library type.
<code>-1 / -2</code>	Paired-end read files.
<code>-p 8</code>	Uses 8 CPU threads.
<code>--validateMappings</code>	Improves mapping accuracy by validating candidate alignments.

These parameters provide a balance between computational efficiency and quantification accuracy.

19.5 SLURM script used for Salmon

```
#!/bin/bash

#SBATCH -J salmon
#SBATCH -o salmon-%j.out
#SBATCH -p medium
#SBATCH --array=2-12%1
#SBATCH --cpus-per-task=8
#SBATCH --mem=50G
#SBATCH --time=7-00:00:00
#SBATCH --mail-type=ALL
#SBATCH --mail-user=gfpeb@ulaval.ca

#####
# 1. Activer entorno
#####
```

```

module load python/3.6
module load htlib/1.8
module load samtools/1.13
module load bowtie/2.4.4
module load jellyfish/2.3.0
module load salmon/1.10.0
module load java/jdk/1.8.0_102
module load trinity/2.15.2
module load rnammer/1.2
module load tmhmm/2.0c
module load signalp/4.1
module load hmmer/3.3
module load ncbiblast/2.10.0
module load transdecoder/5.5.0
module load trinotate/3.2.0

#===== 2. Rutas base =====
base_dir="/project/jcvaguilar/users/jcvil9/Gab_Zamia_Transcri"
reads_base="${base_dir}/01_clean_reads_Zamia_RNA_trimmomatic"

# Archivos originales (Trinity global)
assembly="${base_dir}/03_trinity_separated/Zamia_all_20250902_trinity/Zamia_all_20250902.Trinity"
gene_map="${base_dir}/03_trinity_separated/Zamia_all_20250902_trinity/Zamia_all_20250902.Trinity"

# Carpeta de trabajo
output_base="${base_dir}/20_B_TPM_abun_Zamia_trinity"
mkdir -p "$output_base"

#===== 3. Crear indice Salmon =====
index_dir="${output_base}/salmon_index"
if [[ ! -f "${index_dir}/versionInfo.json" ]]; then
    echo "Creando ndice Salmon en $index_dir"
    mkdir -p "$index_dir"
    salmon index -t "$assembly" -i "$index_dir" -k 31
fi

#===== 4. Seleccionar muestra =====
samples_list="${base_dir}/samples_list.txt"
sample=$(sed -n "${SLURM_ARRAY_TASK_ID}p" "$samples_list" | tr -d '[:space:]')
echo "Procesando muestra: $sample"

left_reads="${reads_base}/${sample}/${sample}_R1_paired.fastq.gz"
right_reads="${reads_base}/${sample}/${sample}_R2_paired.fastq.gz"

out_dir="${output_base}/${sample}_salmon"
mkdir -p "$out_dir"

```

```

##### 5. Ejecutar Salmon #####
salmon quant \
  -i "$index_dir" \
  -l A \
  -1 "$left_reads" -2 "$right_reads" \
  -p $SLURM_CPUS_PER_TASK \
  -o "$out_dir" \
  --validateMappings

echo "Finalizado Salmon para $sample"

##### 6. Generar matrices #####
if [[ ${SLURM_ARRAY_TASK_ID} -eq $(wc -l < "$samples_list") ]]; then
  echo "Generando matrices de abundancia..."
  /prg/trinityrnaseq/2.15.2/util/abundance_estimates_to_matrix.pl \
    --est_method salmon \
    --out_prefix ${output_base}/Zamia_all_TPM \
    --name_sample_by_basedir \
    --gene_trans_map "$gene_map" \
    $(find ${output_base} -name "quant.sf")

  echo "Matrices guardadas en ${output_base}/Zamia_all_TPM.*"
fi

```

19.6 Generation of abundance matrices

After all samples were quantified, Trinity's utility script `abundance_estimates_to_matrix.pl` was used to combine the individual `quant.sf` files into expression matrices.

Two types of abundance matrices were generated:

Matrix	Description
Transcript matrix	TPM estimates for individual transcript isoforms.
Gene matrix	TPM estimates aggregated at the gene level using the Trinity gene-transcript map.

These matrices can be used for downstream analyses such as differential expression, clustering, PCA, co-expression networks, and functional enrichment.

19.7 Output files

For each sample, Salmon generates a quantification directory containing abundance estimates.


```

20_B_TPM_abun_Zamia_trinity/
  salmon_index/

  JCV23-0101_salmon/
    quant.sf
    cmd_info.json
    lib_format_counts.json
    logs/

  JCV23-0102_salmon/
  JCV23-0201_salmon/
  ...

  Zamia_all_TPM.isoform.counts.matrix
  Zamia_all_TPM.isoform.TPM.not_cross_norm
  Zamia_all_TPM.gene.counts.matrix
  Zamia_all_TPM.gene.TPM.not_cross_norm

```

19.8 Interpretation of the main output files

File	Description
quant.sf	Transcript abundance estimates for each sample.
isoform.counts.matrix	Raw counts at transcript level.
isoform.TPM.not_cross_norm	TPM-normalized transcript abundance matrix.
gene.counts.matrix	Raw counts aggregated by gene.
gene.TPM.not_cross_norm	TPM-normalized abundance matrix at gene level.

The TPM matrices generated by Salmon constitute the primary dataset used for downstream transcriptomic analyses because they provide normalized estimates of transcript abundance that are comparable among samples.

20 Final Remarks and Future Analyses

This report describes the complete workflow used to generate, evaluate, filter, annotate, and quantify the *Zamia* transcriptome. Through successive quality-control and contamination-filtering steps, a high-confidence reference transcriptome was obtained and functionally annotated.

Two key products of this pipeline will serve as the foundation for downstream biological analyses:

20.1 Reference Transcriptome

The final transcriptome assembly generated with Trinity and subsequently filtered using coverage and taxonomic criteria provides a curated set of transcript sequences representative of the *Zamia* transcriptome.

The principal files include:

```
Zamia_all_20250902.Trinity.fasta
Zamia_all_20250902.Trinity.fasta.transdecoder.pep
Trinotate_FINAL_04Mar2026.xls
Trinotate_03Mar2026.sqlite
```

These resources contain the assembled transcripts, predicted coding sequences, and functional annotations required for gene characterization and biological interpretation.

20.2 Transcript Abundance Matrices

Transcript abundance was estimated using Salmon by mapping each sample against the global Trinity assembly.

The principal output files include:

```
Zamia_all_TPM.isoform.counts.matrix
Zamia_all_TPM.isoform.TPM.not_cross_norm
Zamia_all_TPM.gene.counts.matrix
Zamia_all_TPM.gene.TPM.not_cross_norm
```

These matrices contain expression estimates at both transcript and gene levels and represent the primary dataset for quantitative transcriptomic analyses.

20.3 Future Analyses

The transcriptome assembly, functional annotation database, and expression matrices generated in this workflow will be used in subsequent analyses, including:

- Differential gene expression analysis between experimental conditions or biological groups.
- Identification of significantly upregulated and downregulated genes.
- Functional enrichment analyses using Gene Ontology (GO) categories.
- Pathway enrichment analyses based on functional annotations recovered through Trinotate.
- Exploration of biological processes, molecular functions, and cellular components associated with transcriptional responses.
- Identification of candidate genes potentially involved in physiological, developmental, or adaptive processes in *Zamia*.

Together, the Trinity assembly, Trinotate annotation, and Salmon abundance estimates provide a robust framework for investigating gene expression patterns and their biological significance in future transcriptomic studies.